



Master's thesis
Master's Programme in Mathematics and Statistics
Statistics

XGBoost for SuperMAG ring current and Pc5 ultra-low frequency wave power indices nowcasting

Jonne Klockars

February 4, 2026

Supervisors: Matti Ala-lahti
Emilia Kilpua

Examiners: Matti Ala-Lahti
Emilia Kilpua
Matti Pirinen

UNIVERSITY OF HELSINKI
FACULTY OF SCIENCE

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki

Acknowledgements

I thank my supervisors for their support and positive attitude throughout the project, especially at the finishing stages. I would not have been able to add relevant space physics and space weather content to this work on my own. We gratefully acknowledge the Super-MAG collaborators (<https://supermag.jhuapl.edu/info/?page=acknowledgement>).

Tiedekunta — Fakultet — Faculty		Koulutusohjelma — Utbildningsprogram — Degree programme	
Faculty of Science		Master's Programme in Mathematics and Statistics	
Tekijä — Författare — Author			
Jonne Klockars			
Työn nimi — Arbetets titel — Title			
XGBoost for SuperMAG ring current and Pc5 ultra-low frequency wave power indices nowcasting			
Työn laji — Arbetets art — Level		Aika — Datum — Month and year	
Master's thesis		February 4, 2026	
		Sivumäärä — Sidantal — Number of pages	
		60	
Tiivistelmä — Referat — Abstract			
Machine learning is a subfield of computer science focused on algorithms that depend on a data set of examples of some phenomenon. Gradient boosting is a machine learning method, which builds models one after another to correct the errors of the previously built model. The XGBoost system used in this thesis is based on gradient boosting. Earth's ring current is an electric current in Earth's magnetosphere at low latitudes. SuperMAG ring current (SMR) index measures the intensity of the current using data from ground magnetometer stations. Pc5 ultra-low frequency (ULF) waves are generated by solar wind in the ionosphere and magnetosphere, and they can also be detected from the ground. Pc5 ULF waves play an important role in transferring solar wind energy to Earth's magnetosphere. In this thesis XGBoost models are developed for nowcasting (forecasting to very near future) SMR and Pc5 ULF wave power indices using data available from Coordinated Data Analysis Web (CDAWeb) maintained by National Aeronautics and Space Administration and SuperMAG, a data repository maintained by organizations and national agencies worldwide. Data was loaded manually from SuperMAG and programmatically from CDAWeb. Two sets of five XGBoost models, one for SMR indices and one for ULF indices, were implemented for nowcasting on different time sectors. Python programming language was used for data processing and for training, evaluating, and testing the XGBoost models. ACM Computing Classification System (CCS): Applied computing → Physical sciences and engineering → Physics Computing methodologies → Machine learning → Machine learning algorithms → Ensemble methods			
Avainsanat — Nyckelord — Keywords			
XGBoost, nowcasting, SMR index, Pc5 ULF waves, solar wind			
Säilytyspaikka — Förvaringsställe — Where deposited			
Muita tietoja — Övriga uppgifter — Additional information			

Contents

Acknowledgements	i
1 Introduction	2
2 Background	4
2.1 Supervised machine learning	5
2.1.1 Model fitting	5
2.1.2 Regularization	7
2.1.3 Performance evaluation	8
2.2 Trees	9
2.2.1 Trees and binary trees	9
2.2.2 Decision trees	10
2.2.3 An example regression tree	11
2.3 Solar wind - magnetosphere coupling	12
2.4 Ring current and Pc5 ultra-low frequency waves	13
3 Methods	15
3.1 Gradient boosting	15
3.2 XGBoost	16
3.2.1 Regularized objective function	17
3.2.2 Objective function optimization	17
3.3 Wavelet transform	19
3.3.1 Fast Fourier transform algorithms	19
3.3.2 Efficient computing of the wavelet transform	21
4 Data	23
4.1 SuperMAG	23
4.2 Coordinated Data-Analysis Web	24
4.3 Data processing	25
5 Method implementation	28

5.1	Feature engineering	28
5.2	XGBoost hyperparameters	29
5.3	Model performance evaluation	31
6	Results	32
6.1	SMR indices	32
6.2	SMR models' feature importance	33
6.3	Pc5 indices	36
7	Discussion and future research	38
	Bibliography	41
	Appendix A Mathematics	49
A.1	Taylor polynomial	49
A.2	Complex conjugate	49
A.3	Convolution theorem	49
	Appendix B Performance metrics	51
	Appendix C Neural Networks and Deep Learning	53
	Appendix D Feature importance and SHAP summary	54
D.1	XGBoost feature importance	54
D.2	Shapley Additive Explanations	58

1. Introduction

Machine learning refers to a subfield of computer science which is focused on developing algorithms dependent on a data set of examples of some phenomenon. The examples can be measurements of a natural phenomenon, or created by humans or algorithms. Machine learning can also be defined as a process of solving a problem with a dataset and an algorithmically built statistical model based on that dataset. Machine learning can be coarsely divided into three types: supervised learning, unsupervised learning, and reinforcement learning. [17]

Machine learning is a powerful tool for analyzing large and complex data sets across a wide range of fields. In recent years its usage has grown rapidly in space physics which is a discipline that studies the Sun, solar wind, heliosphere and planetary magnetospheres and upper atmospheres within the Solar System [107]. In this context machine learning has been used for example in pattern recognition, clustering, time series analysis, spatio-temporal prediction, and knowledge discovery [13] with one key application being space weather forecasts [18]. Space weather phenomenon include solar magnetic activity and its effects in Earth's magnetosphere, ionosphere and the atmosphere [107]. Advanced technology, which the global society depends on, is vulnerable to space weather disturbances that can cause GPS positioning errors, disrupt satellite communications, and shut down electric power-grids [103]. Performance of the current space weather forecasting models is quite modest [11] [58]. It is important to develop better models so that the potential disturbances caused by varying space weather can be mitigated or avoided with accurate forecasts.

Gradient boosting is a machine learning method which builds an ensemble of models one after another, each correcting the errors of the previous model [30]. XGBoost is a scalable implementation of gradient boosting, used widely in supervised learning tasks [21]. According to the World Meteorological Organization, nowcasting is defined as forecasting from the present moment up to six hours ahead [111]. In this thesis XGBoost models are developed for SuperMAG ring current (SMR) index and Pc5 ultra-low frequency (ULF) wave power nowcasting. The models are trained on solar wind measurements upstream of the Earth and the previous values of SMR index and Pc5 ULF wave power obtained from SuperMAG [98]. Python programming language [109] is used in loading and processing

the data, implementing XGBoost models, and producing plots of the results. All of the code is accessible in GitHub [60].

The thesis is structured into seven chapters, this introduction being the first. Chapter two contains background information on supervised machine learning, decision trees, solar wind, and magnetosphere. Theoretical aspects of the main methods - gradient boosting, XGBoost, and wavelet transform - are presented in chapter three and the fourth chapter describes the data sources and data processing. Implementation of the methods is presented in chapter five and results in the sixth chapter. The last chapter contains discussion and directions for future work.

2. Background

The goal in supervised machine learning is to build a model which outputs a prediction or a label based on the input data. For training the model, a data set of examples including the true values or labels for the predictions is required in contrast to unsupervised learning, where the algorithm's task is to analyze or cluster the data without access to labels or other true values [51]. Reinforcement learning is focused on developing models by rewarding or punishing them based on their actions in some particular environment [87]. In machine learning literature the known true values are called the ground truth, predicted variables are called targets, and the variables used for the predictions are called features. The example data is usually split into three sets: training set, validation set, and testing set. A training set is used by the model algorithm to learn patterns from the data and a validation set is used for validating its performance. Validation data can be used to select a suitable machine learning algorithm and for tuning model parameters. Once a satisfactory level of performance has been achieved, the model is tested on the previously unused test set. Since the test set has not been used in model selection or parameter tuning, it gives an unbiased estimate of the model's performance. If the same set would be used for validation and testing, the model would be biased towards performing well on that set and the reported performance metrics would be overly optimistic. [17]

A common approach in the past has been to use 70% of the data for training, 15% for validation and the remaining 15% for testing. Currently, as the volume of data in most fields has significantly increased, a data set may contain millions of samples. For such data, a 95%/2.5%/2.5% split is as feasible as a 70%/15%/15% split. There is no optimal split but usually most of the example data is in the training set which is the one used for building the model. [17]

Section 2.1 introduces supervised machine learning in a slightly more detailed manner. Sources used for that section are the fourth edition of *Artificial Intelligence: A Modern Approach, Global Edition* by Russell and Norvig [92] and *The Hundred-page Machine Learning Book* by Burkov [17]. Trees are discussed in section 2.2, solar wind - magnetosphere coupling is briefly described in section 2.3, and Earth's ring current and Pc5 ultra-low frequency waves in 2.4.

2.1 Supervised machine learning

Let $f : \mathbb{C}^n \rightarrow \mathbb{C}^m$, $f(x_i) = y_i$ be an unknown function and

$$(x_i, y_i) \in \mathcal{D} = \{(x_i, y_i) \mid i \in \{1, 2, \dots, N\}\}, \quad (2.1)$$

where y_i are the targets and x_i are the features of an example data set $\mathcal{D}$. Complex-valued models exist for example in the field of signal processing [67] while real-valued representations are sufficient for images, text, and tabular data. Supervised learning can be defined as trying to construct the best possible approximation for f , denoted here by h , $h(x_i) = \hat{y}_i$, where $\hat{y}_i$ is an approximation for target y_i . The function h can be thought also as a hypothesis about the world or a model of the data. The goal is to construct an h which performs well also on new data not included in the training set $\mathcal{D}_{train}$. The attempt at programming such an h happens by optimizing an objective function, which typically includes minimizing the mean of a loss function $l(y, h(x))$ over the training data:

$$\frac{1}{N} \sum_{(x,y) \in \mathcal{D}_{train}} l(y, h(x)), \quad (2.2)$$

A loss function describes the difference between the values predicted by the model and the ground truth. Examples of loss functions include absolute value loss $l_1(y, \hat{y}) = |y - \hat{y}|$, squared error loss $l_2(y, \hat{y}) = (y - \hat{y})^2$, and error rate loss $l_{0/1}(y, \hat{y}) = 0$ if $\hat{y} = y$, else 1, for classification tasks with discrete outputs. In addition to loss, an objective function can have terms penalizing for model complexity and terms related to the particular data, domain, or task. XGBoost, for example, uses a regularization component in its objective function (Equation 3.2) for penalizing more complex models in order to build simpler and effective ones.

In addition to parameters learned from the data for optimizing the objective function (regression coefficients for example), a machine learning model has also external parameters which it does not learn from the data. These parameters are called hyperparameters and their values are defined by the developer. Hyperparameters control the model's learning process. The process of choosing hyperparameter values is called hyperparameter tuning. Hyperparameters are usually tuned iteratively for optimizing model performance after choosing the right kind of model for the data and the task. Hyperparameters of the XGBoost models used in this work are discussed in Section 5.2.

2.1.1 Model fitting

A machine learning model is said to underfit if it does not perform well on the training data set. An underfitting model does not find meaningful patterns from the data and it is said to have a high bias. In this context bias means a tendency to deviate from the

expected value when model performance is averaged over different training sets. The main two causes for underfitting are a model which is too simple and uninformative features. For example, a linear model can underfit nonlinear data. Building a more complex model which is able to learn useful patterns from the data and engineering features that lead to better predictions are the solutions to these two problems. In space weather forecasting context this could mean using combinations of the measured variables, for example the product of solar wind speed and proton density, as additional features. Feature engineering for this project is discussed in Section 5.1.

A model performing well on the training data, but poorly on previously unseen data, is overfitting. An overfitting model is said to have a high variance. Variance in this context means the amount of change in the model caused by fluctuation in the data. Predictions from a high variance model change significantly with the input data. This kind of a model focuses too much on the details of the training data and fails to generalize for predicting from unseen data. Lower variance models are more likely to provide more stable predictions and perform better with unseen data, but they may also be too simple and underfit. One of the main causes of overfitting is excessive model complexity. An overly complex model may achieve perfect performance on the training set by memorizing the training data, but perform poorly with unseen data which deviates from the training data patterns. For example, very deep trees (Section 2.2) are prone to overfitting. Another common cause is having a too small data set. Even a simple model might learn noise or distorted relations between features of a small training set while trying to find something to build predictions upon, resulting in poor generalization outside of the training data.

Finally, having too many features and too few training examples may result in what is known as the curse of dimensionality [8] or peaking phenomenon, also known as the Hughes phenomenon [48]. These terms are sometimes used interchangeably, even though the peaking phenomenon is a description of classifier behavior and the curse of dimensionality is a more general pattern observed with high-dimensional data. The dimensionality of a dataset having a fixed number of samples is determined by the number of its features. The curse of dimensionality in machine learning context refers to four issues related to having too many features. Firstly, data becomes sparser and sparser in $\mathbb{R}^d$ as the feature dimension d of the data grows, and models may struggle or completely fail to find patterns or groups from sparse data. Secondly, the ratio of distances between any two points approach one in high-dimensional spaces (there are some metrics for which this does not hold, though) degrading the performance of distance-based methods [3] [10]. Thirdly, too many features may result in overfitting. The fourth issue is an increased demand for computational resources to process and store more complex data. Hughes phenomenon refers to the average performance of a classifier increasing with the number of features to a certain dimensionality, and declining as the number of features grows

more [106].

Solutions for these common causes of overfitting are a reduction of the model complexity, reducing the dimensionality of the data, and using more data. Using more data will not help, if all the data is noisy. Methods for reducing model complexity include choosing a simpler type of machine learning algorithm and model regularization discussed below.

2.1.2 Regularization

Regularization refers to all the techniques used for forcing the learning algorithm to build a less complex model. Regularization often leads to a significant reduction of variance but slightly higher bias. This is called bias-variance tradeoff. In other words, regularization reduces the risk for overfitting but excessive regularization can lead to underfitting. The two most common regularization methods are lasso (L1) [104] and ridge (L2) [42] regularization. Combination of both is called elastic net regularization. Lasso adds the sum of the absolute values of model parameters θ to the objective function while ridge adds the sum of squares of θ . The sums can be weighted by a hyperparameter.

For an example, we can consider Equation 2.2 as an objective function on its own. Regularizing it with L1 would look like this

$$C_1 \sum_{i=1}^N |\theta_i| + \left(\frac{1}{N} \sum_{(x,y) \in \mathcal{D}_{train}} l(y, h(x)) \right), \quad (2.3)$$

and similarly L2 regularization would look like

$$C_2 \sum_{i=1}^N \theta_i^2 + \left(\frac{1}{N} \sum_{(x,y) \in \mathcal{D}_{train}} l(y, h(x)) \right). \quad (2.4)$$

C_1 and C_2 are hyperparameters adjusting the importance of regularization.

In the case of a linear regression model, θ represents the weight assigned to each input feature. For example, if a regression model predicts

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2, \quad (2.5)$$

then θ_1 and θ_2 are the parameters that using L1 or L2 penalties would shrink (most algorithms do not include the intercept term θ_0 in regularization [38]): as the model attempts to minimize loss, it adjusts these parameters to smaller values, which in turn helps prevent overfitting. As the parameter values θ_1 and θ_2 would shrink, the amount of fluctuation caused by different values of features x_1 and x_2 would decrease. L1 can also push parameters to zero which results in not using the associated features making the model simpler.

2.1.3 Performance evaluation

The most simple way of assessing the performance of a machine learning model is to look at the predictions based on the unseen test set. If the model predicts the targets of the examples from the test set well, then the model is said to generalize well. Another way of expressing the good generalization ability of the model is to simply state that the model performs well. [17]

Various formal metrics can be used, too. For example root mean squared error, mean absolute error, median absolute error and scaled versions of them (Appendix B) for regression tasks as in this work. A significantly lower error on the training data compared to the test data is a sign of overfitting. The meaning of 'significantly' depends on the task, there is no fixed rule for determining it. [17]

Cross-validation techniques are widely used for machine learning model performance evaluation. A common practice, called k -fold cross-validation, is to split the training data into k sets. A model is trained k times using a different one of the k sets as the validation set and all the others as the training set each time. Performance metrics are averaged over all the k training - validation splits and hyperparameters are tuned based on the average performance. After reaching satisfactory performance, all the training data is used to train a model which is tested on the test set. This is a good strategy when the available data set is small because it doesn't use a separate validation set. The data is split into two sets, training set and testing set, instead of three, and thus more data can be used for training the model. A variation of k -fold cross-validation does not use a separate test set so all the data can be used for training the model and the average metrics are the final test for evaluating the model parameters. [17] [54]

There is an obvious problem with the k -fold cross-validation described above if the order of the samples is meaningful or if any samples are dependent on preceding values. Using random splits mixes up the order of the data and in the case of a time series, for example, the model ends up predicting the past from the future. In time series cross-validation, used also in this project, the data is split into consecutively larger and larger sets each starting from the beginning of the time series and using data from the future for validation. For an example, consider a time series with four samples: $\{x_0, x_1, x_2, x_3\}$. A time series cross-validation could be performed with this data by having three training and three validation sets. The training sets could be $\{x_0\}$, $\{x_0, x_1\}$, and $\{x_0, x_1, x_2\}$. The first training set $\{x_0\}$ could be validated with $\{x_1\}$, the second training set $\{x_0, x_1\}$ could be validated with $\{x_2\}$ and the third training set $\{x_0, x_1, x_2\}$ could be validated with $\{x_3\}$. This way the initial values of the data will never be used for validation and the model will always predict future targets from past features. The exact training and validation sets are based on the data and the task. For example, particular periods can be chosen

as the validation sets if there is a reason to believe that it will improve performance on unseen data. [86]

2.2 Trees

In computer science, a tree refers to an abstract data structure introduced in this section to the extent relevant for this thesis. XGBoost uses a specific type of trees called classification and regression trees.

2.2.1 Trees and binary trees

Everything in this subsection can be found from Donald Knuth's book *The Art of Computer Programming Volume 1* [61]. A tree can be recursively defined as a finite set T of one or more nodes such that there is one node called the root, denoted here by $\text{root}(T)$, and all the other nodes are partitioned into $n \geq 0$ disjoint sets $T_1, \dots, T_n$ so that T_m is a tree for all $m \in \{1, \dots, n\}$. These trees $T_1, \dots, T_n$ are called the subtrees of $\text{root}(T)$. The degree of a node equals the number of subtrees of the node. A node with degree zero is called a terminal node or a leaf. The level of a node with respect to the whole tree T is defined recursively: the level of $\text{root}(T)$ is zero, and the level of any other node is one higher than the node's level with respect to the subtree of $\text{root}(T)$ containing it. The depth of a tree is defined to equal the maximum level of its nodes. Sometimes height is used instead of depth when referring to the maximum level of a tree's nodes.

In a binary tree each node has at most two subtrees. If a binary tree has only one subtree, then the left and right subtree are distinguished from each other. A binary tree is defined recursively to be a finite set of nodes that is empty, or contains a root and the elements of two disjoint binary trees called the left and right subtrees of the root.

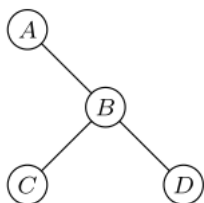


Figure 2.1: The most common approach to visualizing a binary tree.

The most common way to visualize a binary tree is depicted in Figure 2.1. The root is placed on top and the tree is grown downwards. The root of binary tree T in Figure 2.1 is A . It has one non-empty subtree T_1 (right subtree) with $\text{root}(T_1) = B$. T_1 has two non-empty subtrees T'_1 and T'_2 , $\text{root}(T'_1) = C$ and $\text{root}(T'_2) = D$. T'_1 and T'_2 do not have non-empty subtrees. The leaves of T are C and D , and its depth (height) is two.

2.2.2 Decision trees

This subsection is based on the book *An Introduction to Statistical Learning with Python* by Gareth James et al. [54]. Prediction approaches involving a split of the feature space into disjoint sets are known as decision tree methods because the splitting rules used for the splitting can be summarized as a tree. Decision tree methods can be applied to, for example, classification and regression tasks. Sample features $\mathbf{x}$ are thought here as vectors, but any suitable data structure can be used.

Building a regression tree can be divided into two steps. The first is to sort all $\mathbf{x}$ in the training data into n disjoint sets X_i , $i \in I$, and $|I| = n$. The second step is to set the predicted value for each $\mathbf{x}$ in a given X_i to be the mean of the target values of $\mathbf{x}$ in that set. The goal of the first step is to find sets X_i that optimize an objective function. That is, for example, minimizing the residual sum of squares

$$\sum_{i \in I} \sum_{\mathbf{x} \in X_i} (y - \hat{y}_{X_i})^2, \quad (2.6)$$

where $\hat{y}_{X_i}$ is the mean of target values of features in X_i , $\mathbf{x}$ is a feature vector, and y is the true target value of the feature vector $\mathbf{x}$.

It is usually practically impossible to compute every possible partition of the features and compare the results. A procedure called recursive binary splitting is used instead. It begins at the root of the tree having all training features in a single set and splits them to two new sets based on a splitting condition. After the first split, splits are made in the two new sets and so on. For a split, a feature x_j and a value v present in the training data are chosen so that splitting the features into $\{\mathbf{x} | x_j < v\}$ and $\{\mathbf{x} | x_j \geq v\}$, leads to the greatest possible gain in terms of the objective function. Considering all x_i and all v for each x_i can become computationally infeasible for data with a large number of features having values from a wide range so approximating algorithms for finding the splits are sometimes used. For approximate algorithms, see for example [21], [36], and [102].

At each node, the best split is made based solely on the current step. This is done without considering the effect on future splits, which could lead to a better tree with a different choice at the current step. This kind of approach is called greedy in computer science. The process goes on until a stopping condition is reached. A condition for stopping the algorithm can be, for example, that all sets contain less than a set amount of training samples or that the tree has reached a certain depth. At the end of the process the algorithm has built a set of rules, based on which it assigns a new feature vector given to it into one of the sets X_i . The sets X_i are the leaves of a binary tree. The average of the training targets of training features in that X_i is the prediction for the new observation. Decision trees can become too complex and overfit the data. The regularization written into the XGBoost algorithm is presented in Section 3.2, and in

Section 5.2 further regularization introduced by the software implementation of XGBoost is discussed.

It might sometimes be useful to use splits into more than two sets at each step. However, it is not a good general strategy because such splitting usually fragments the data too quickly. As a result, the following splits become increasingly unreliable because of having a decreasing number of samples to base the decisions on. The effect of splitting into more sets can be achieved by a series of binary splits which are based on more statistical evidence, evaluated by the algorithm's objective function. For example, consider a data set of 1000 samples. Using a binary split, there are 500 samples in both new sets while a 5-way split results in having 200 samples in each new set after the initial split. One level down, binary splitting results in having 250 samples in each new set to base a possible further splits on while the 5-way splitting has reduced the sample size of each new set to 40. [38]

Classification and regression trees are built in a very similar way. The difference is, that a regression tree predicts the mean of training data target values in a leaf, while a classification tree predicts the most common target (class) of the training samples in a leaf. Another difference is the loss computation. Regression loss functions are not suitable for computing a classification loss. The nowcasting tasks in this thesis are regression problems so classification trees are not discussed further. The motivation for mentioning them is a special case of a binary tree, classification and regression tree (CART) used in XGBoost. CART was developed by Leo Breiman, Jerome Friedman, Richard Olshen, and Charles Stone. Their book *Classification and Regression Trees* [16] was published in 1984. In short, CARTs can handle both classification and regression tasks using the methods described in this section in addition to other techniques. The details can be studied from the book.

2.2.3 An example regression tree

As an example, consider a set of five samples with three features x_1 , x_2 , x_3 , and a target y :

Sample	x_1	x_2	x_3	y
1	2	5	1	10
2	3	6	4	12
3	5	7	2	20
4	7	2	5	18
5	9	4	3	25

Suppose the algorithm first splits using feature x_1 and a threshold value of 5. The feature space is now divided into two sets:

$$X_1 = \{(x_1, x_2, x_3) : x_1 \leq 5\} = \{1, 2, 3\}, \quad X_2 = \{(x_1, x_2, x_3) : x_1 > 5\} = \{4, 5\}.$$

Next, within X_1 a further split could be made, for example, using $x_2 \leq 6$:

$$X_3 = \{(x_1, x_2, x_3) : x_1 \leq 5, x_2 \leq 6\}, \quad X_4 = \{(x_1, x_2, x_3) : x_1 \leq 5, x_2 > 6\}.$$

The samples would now be partitioned into three non-overlapping sets:

$$X_3 : \{1, 2\}, \quad X_4 : \{3\}, \quad X_2 : \{4, 5\}.$$

The means of target values in each set would be

$$\hat{y}_{X_3} = \frac{10 + 12}{2} = 11, \quad \hat{y}_{X_4} = 20, \quad \hat{y}_{X_2} = \frac{18 + 25}{2} = 21.5,$$

and the residual sum of squares (equation 2.6) computation would be

$$\begin{aligned} & \underbrace{(10 - 11)^2 + (12 - 11)^2}_{X_3} + \underbrace{(20 - 20)^2}_{X_4} + \underbrace{(18 - 21.5)^2 + (25 - 21.5)^2}_{X_2} \\ &= (1^2 + 1^2) + (0^2) + ((-3.5)^2 + (3.5)^2) \\ &= (1 + 1) + 0 + (12.25 + 12.25) = 2 + 0 + 24.5 = 26.5. \end{aligned}$$

A visualization of the example regression tree is presented in figure 2.2 below.

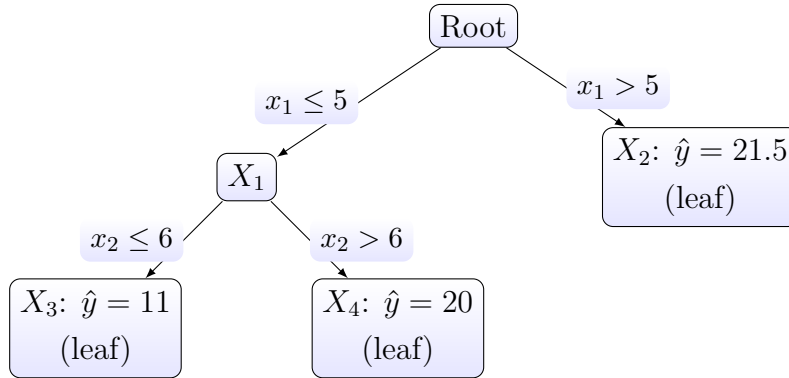


Figure 2.2: Example regression tree.

2.3 Solar wind - magnetosphere coupling

Solar wind is a continuous stream of charged particles, primarily electrons and protons, originating from the sun predicted by Parker in 1958 [85]. During the Luna 1 mission in 1959, a group of Russian scientists made the first observations of a supersonic flow

in interplanetary space. The existence of the continuous solar wind phenomenon was confirmed by the Mariner 2 spacecraft, launched in 1962, while it was en route to Venus. [52]

Solar wind observed by spacecraft orbiting the Lagrangian L1 point (Section 4.2) propagates to the Earth in approximately 30 to 120 minutes [89]. Various models have been created to estimate the delay from the measurements at the L1 point to the Earth's magnetosphere, the most simple ones being a constant one hour estimate and a simple vector model based on solar wind speed at the L1 point [71]. More complex and accurate estimates include various machine learning implementations and models based on physics [81] [89].

Interaction between the solar wind and the magnetic field of Earth creates a hollow around the planet known as the magnetosphere [20]. The magnetosphere reaches about 10 Earth radii out into space on the dayside, and on the nightside it is elongated to a magnetotail of more than 100 Earth radii [55]. The Earth's magnetosphere is coupled with the Earth's ionosphere, which is partially ionized gas between space and the Earth's lower atmosphere [57] located approximately between 80 and 650 kilometers above the ground [75]. There are several electric current systems in the magnetosphere and in the ionosphere [57]. Considering this work, the most important current is the ring current circulating the Earth at about two to nine Earth radii above the equator [23]. Geomagnetic storms are disturbances in the Earth's magnetosphere caused by, for example, fast solar wind streams during which the ring current significantly intensifies [57].

The activity in the magnetosphere changes as the properties of the solar wind change. Thus the solar wind drives geomagnetic activity in Earth's magnetosphere-ionosphere system. This dynamic interaction is called the solar wind-magnetosphere coupling. Space weather phenomena, including geomagnetic storms and auroras, dependent on these interactions are monitored by indices like SMR and Pc5 ULF studied in this thesis. [12]

The properties of the solar wind are not constant but vary over different timescales, the most fundamental being the approximately 11-year solar cycle driven by changes in the Sun's magnetic activity and coronal structure. This cyclical variation affects the global parameters such as speed, density, and magnetic field strength, which in turn modulate geomagnetic activity levels at Earth. [39]

2.4 Ring current and Pc5 ultra-low frequency waves

Newell and Gjerloev describe the Earth's ring current as 'the energy of suprathermal ions circulating about the Earth', and write that this 'circulation is produced by energization of plasma sheet ions by solar wind driving' [80]. When plasma sheet ions and electrons are injected into ring current region, ions start drifting westward and electrons eastward

around the Earth [27]. Despite its name, the ring current is not symmetric. It exhibits a varying dawn-dusk asymmetry pattern and it is currently believed that plasma injections and the resulting drifts, decay processes, asymmetric solar wind-magnetosphere coupling, and certain current systems are among the key causes behind the asymmetries [84]. In particular, ring current is asymmetric during strong magnetospheric storms, when ions and electrons complete only partial drift before being lost at the dayside magnetopause [49]. The ring current, and thus the SMR index which measures the intensity of the current, is a good indicator of disturbances in Earth's magnetosphere and the related space weather effects [34]. A set of five XGBoost models was developed in this thesis for nowcasting the SMR indices.

Large-scale oscillations in magnetosphere's plasma and magnetic field composition, generated by solar wind interacting with the Earth's magnetosphere, are called ultra-low frequency waves. They are a major solar wind energy pathway into the Earth's magnetosphere [74]. The lowest continuous wave frequency range, 2-7 mHz, of these 2-5000 mHz oscillations is the target of another set of five nowcasting XGBoost models presented in this thesis. Waves at this frequency are called Pc5 ULF waves [53].

The interest to ULF waves stems from them being one of the major contributors to acceleration and transportation of high-energy charged particles within Earth's radiation belts [72]. The radiation belts are toroidal regions approximately co-located with the ring current but containing much higher energy particles [7]. These charged particles are dangerous to satellites in Earth's orbit in this domain [97].

3. Methods

This thesis work focuses on developing XGBoost models for space weather nowcasting. Model development includes processing data used in space physics research and operative space weather services, including a wavelet transform computation of ground magnetic field measurements. Theoretical aspects of XGBoost and wavelet transform are explained in this chapter. An introduction to gradient boosting is also presented as a wider theoretical frame for XGBoost. Implementation of the XGBoost algorithm and the wavelet transform are discussed in Chapter 5. *XGBoost: A Scalable Tree Boosting System* by Chen and Guestrin [21], and the related open source software package documentation [43] are used as a source for Section 3.2. The main source for Section 3.3 is *A Practical Guide to Wavelet Analysis* by Torrence and Compo [105] and *Discrete-Time Signal Processing* by Oppenheim and Schaffer [83] is used for Subsection 3.3.1

3.1 Gradient boosting

Gradient boosting refers to a technique in which low-capacity machine learning models are build one after another, each model optimized to correct the errors of the previous one, and the final result is combined from all the individual models [30]. 1988 machine learning course notes by Michael Kearns archived as *Thoughts on Hypothesis Boosting* [56] are one of the earliest discussions about boosting. Kearns built on Valiant’s probably approximately correct (PAC) learning framework [108] and asked if an efficient weak learner implies an efficient learner of arbitrary accuracy. Weak learner refers to Valiant’s weak learner which is a classification algorithm performing only slightly better than random guessing. Kearns proposed that the accurate learner could be obtained through iterating the weak learner and using higher weights for misclassified examples.

Freund and Schapire published the AdaBoost algorithm [29] in 1997. It was the first practical implementation of the boosting ideas [63]. Breiman set AdaBoost and other algorithms into adaptive reweighting and combining (ARC) statistical framework [14] [15]. Other researchers developed the statistical framing further by connecting it to loss functions, additive modeling, logistic regression and finally regression [63]. Friedman’s gradient boosting [30] and stochastic gradient boosting [31], published in 2001 and 2002,

reframed boosting as a general gradient descent algorithm in function space and created a unified theoretical framework in which arbitrary differentiable loss functions could be minimized. In addition, the stochastic approach reduced overfitting and computation time in applications [38]. XGBoost [21], LightGBM [37], and CatBoost [88], are three examples of modern methods built on Friedman’s ideas [9].

To summarize, a gradient boosting algorithm attempts to build a strong model by sequentially adding weak models. These concepts do not have formal theoretical definitions. They are inspired by, but not identical to, the PAC framework’s strong and weak learner. PAC’s strong learner is an algorithm which can learn a hypothesis with a high probability for a small error in a reasonable time, formally defined in [93]. Each new model is fit to the negative gradients of the loss function with respect to the current model’s predictions. The gradients indicate the direction in which the predictions should be adjusted to reduce the loss function value most efficiently. Thus, the algorithm focuses on regions where the model’s errors are most significant, iteratively improving its performance. Typical initialization uses the best constant guess, for example the mean, of the predicted values and computes the gradients. At each step, gradients are computed and a new weak model is fit to these gradients and added to the ensemble. This procedure continues until a stopping condition, for example the error not decreasing in certain amount of rounds or maximum number of models reached, is met. Trees can be used as weak models by simply restricting their depth. Predictions from separate trees can easily be added together to generate a new, more efficient model. In addition, trees can be generated very quickly. These properties make them an excellent choice for boosting. [63]

3.2 XGBoost

XGBoost stands for eXtreme Gradient Boosting. It is a machine learning system based on gradient boosting used for supervised learning tasks. The most important innovations of the method were its scalability, efficiency with sparse data, parallel and distributed computing, and out-of-core-computing in addition to inbuilt regularization of the objective function. Out-of-core-computing refers to methods designed to process data too large to fit the computer’s main memory (RAM) at once. Instead of loading the entire data, out-of-core methods process smaller sets of the data sequentially [110]. The objective function and its optimization are presented here as they relate directly to the nowcasting tasks but the other aforementioned innovative approaches are not described in this thesis as they are more of a topic for a computer science presentation.

3.2.1 Regularized objective function

XGBoost ensembles classification and regression trees by summing functions representing the CARTs one by one. Given a data set $\mathcal{D} = \{(x_i, y_i) \mid x_i \in \mathbb{R}^m, y_i \in \mathbb{R}^k\}$, $|\mathcal{D}| = n \in \mathbb{N}$, where x_i are the features and y_i the target, an XGBoost model uses K functions to predict the output

$$\hat{y}_i = \phi(x_i) = \sum_{k=1}^K f_k(x_i), f_k \in \mathcal{F}, \quad (3.1)$$

where $\mathcal{F} = \{f(x) = w_{q(x)} \mid q : \mathbb{R}^m \rightarrow \{1, 2, \dots, T\}, (w_1, w_2, \dots, w_T) = \vec{w} \in \mathbb{R}^T\}$ is the set of all CARTs, q a mapping (decision rules) from a data point x to a leaf index, and T is the number of leaves. Each f_k is a function of independent tree structure q and leaf weights $w_{q(x_i)}$. Each regression tree leaf contains a continuous score w_i , $i \in \{1, 2, \dots, T\}$. The functions f_k are learned by optimizing a regularized objective function

$$\mathcal{L}(\phi) = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^t \Omega(f_k), \quad (3.2)$$

where $\Omega(f) = \gamma T + \frac{1}{2} \lambda \|\vec{w}\|^2$,

l is a differentiable convex function measuring the difference between the prediction $\hat{y}_i$ and the target y_i , and t is the number of trees. The first term of the regularization function Ω grows with the number of tree leaves T adjusted by hyperparameter γ , and the second term is the L1 (Subsection 2.1.2) term adjusted by hyperparameter λ . Ω penalizes complex trees more than simple ones. As a result, optimizing $\mathcal{L}$ tends towards building simple and predictive trees.

3.2.2 Objective function optimization

Optimization of the tree ensemble model $\mathcal{L}$ (Equation 3.2), which has discrete non-differentiable tree structure mappings as parametres, is done by adding one tree at a time. Writing the prediction from the i -th data point at the k -th step as $\hat{y}_i^{(k)}$ gives the following objective for the k -th step:

$$\mathcal{L}^{(k)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(k-1)} + f_k(x_i)) + \Omega(f_k). \quad (3.3)$$

Mean squared error as the loss function l yields a closed form easily, but obtaining closed forms for other loss functions is not simple [24]. For the general case, a second-order Taylor polynomial (Appendix A.1) of the loss function l is used, resulting in

$$\mathcal{L}^{(k)} \approx \sum_{i=1}^n [l(y_i, \hat{y}_i^{(k-1)}) + g_i f_k(x_i) + \frac{1}{2} h_i (f_k(x_i))^2] + \Omega(f_k), \quad (3.4)$$

where $g_i = \partial_{\hat{y}_i^{(k-1)}} l(y_i, \hat{y}_i^{(k-1)})$ and $h_i = \partial_{\hat{y}_i^{(k-1)}}^2 l(y_i, \hat{y}_i^{(k-1)})$. Second-order form makes the loss quadratic, meaning faster convergence compared to a loss using only the gradient [82]. The loss terms from previous steps are constant with respect to the k -th step so they do not affect the k -th minimum. Discarding them yields a simplified general objective function for step k :

$$\begin{aligned}\bar{\mathcal{L}}^{(k)} &= \sum_{i=1}^n [g_i f_k(x_i) + \frac{1}{2} h_i (f_k(x_i))^2] + \Omega(f_k) = \\ &= \sum_{i=1}^n \left[g_i w_{q(x_i)} + \frac{1}{2} h_i (w_{q(x_i)})^2 \right] + \gamma T + \frac{1}{2} \lambda \|\vec{w}\|^2 = \\ &\quad \sum_{j=1}^T \left[\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T,\end{aligned}$$

where $I_j = \{i \mid q(x_i) = j\}$ is the set of data point indices assigned to the j -th leaf. The index of the summation in the third line has been changed because all the data points on the same leaf get the same score. Writing $G_j = \sum_{i \in I_j} g_i$ and $H_j = \sum_{i \in I_j} h_i$ gives

$$\bar{\mathcal{L}}^{(k)} = \sum_{j=1}^T [G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2] + \gamma T. \quad (3.5)$$

Now for a fixed $q(x)$ and leaf j , the optimal weight w_j^* is found by minimizing $\bar{\mathcal{L}}^{(k)}$ with respect to w_j :

$$\frac{\partial \bar{\mathcal{L}}^{(k)}}{\partial w_j} = G_j + (H_j + \lambda) w_j = 0 \implies (H_j + \lambda) w_j = -G_j \implies w_j^* = -\frac{G_j}{H_j + \lambda}. \quad (3.6)$$

So the optimal objective function for step k is

$$\bar{\mathcal{L}}^{(k)*} = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T. \quad (3.7)$$

The optimal objective function $\bar{\mathcal{L}}^{(k)*}$ can be used as a scoring function for a tree structure q : the smaller the value, the better the tree. It is usually not possible to iterate all possible trees so a greedy algorithm which optimizes one level of the added tree f_k at a time is used instead. The following equation gives the loss reduction after a leaf is split into two new leaves.

$$\bar{\mathcal{L}}^{(k)} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma. \quad (3.8)$$

The first term in the brackets is the score of the new left leaf, the second term is the score of the new right leaf, and the third is the score of the original leaf. If the scores of new left and right leaves are not high enough, then the split is not executed as the loss reduction would be negative, thus adding to the loss instead of reducing it. After the last split of f_k , the algorithm moves on to optimizing the next tree f_{k+1} .

3.3 Wavelet transform

The wavelet transform introduced by Morlet in his 1983 paper *Sampling Theory and Wave Propagation* [73] has been widely used in various fields of science: image processing, seismic signal detection, optics, medical research, and quantum mechanics, for example. It is suitable for studying multiscale nonstationary processes occurring over finite spatial and temporal domains. [65]

Wavelet transform requires an evenly spaced time series and a function ψ_0 depending on a nondimensional time parameter η . In addition, $\psi_0(\eta)$ must have a mean of zero and it must be zero or arbitrarily close to zero outside of some bounded interval in both frequency and time domains [26]. The Morlet wavelet function

$$\psi_0(\eta) = \pi^{-1/4} e^{i\omega_0\eta} e^{-\eta^2/2}, \quad (3.9)$$

where $\omega_0 = 6$ so that it meets the requirements [26], is used in this thesis following Lipsanen [68]. The continuous wavelet transform on a discrete sequence $\{x_n | n \in \{0, 1, \dots, N-1\}\}$ for time index n and scale s is defined as

$$W_n(s) = \sum_{n'=0}^{N-1} x_{n'} \psi^* \left(\frac{(n' - n)\delta t}{s} \right), \quad (3.10)$$

where ψ^* is the complex conjugate (Appendix A.2) of the scaled and translated ψ_0 and δt is sampling interval.

The wavelet power spectrum P is defined as the square of the magnitude of the wavelet transform, $P = |W_n(s)|^2$, and the scale averaged wavelet power

$$\overline{W_n^2} = \frac{\delta j \delta t}{C_\delta} \sum_{j=j_1}^{j_2} \frac{|W_n(s_j)|^2}{s_j} \quad (3.11)$$

is the weighted sum of the wavelet power spectrum over scales from s_1 to s_2 . It is used for examining fluctuations in power over a range of scales. This averaging function, implemented with PyCWT (Section 4.3) in the Pc5 period scale range of 150-600 seconds was used for computing the Pc5 ULF wave power indices from SuperMAG magnetic field data for the nowcasting task using the parameters defined by Helsinki University Space Physics Group [41].

3.3.1 Fast Fourier transform algorithms

The discrete Fourier transform of a time series $\{x_n | n \in \{0, 1, \dots, N-1\}\}$ is defined as

$$\hat{x}_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N}, \quad (3.12)$$

where $k \in \{0, 1, \dots, N-1\}$ is the frequency index. [95]

The efficient computing of the wavelet transform described in the next subsection utilizes a fast Fourier transform (FFT) algorithm. Algorithms which divide a problem into smaller subproblems of the same type, solve them independently, and combine the solutions to solve the original problem, are called divide-and-conquer algorithms. Most FFT algorithms are divide-and-conquer algorithms based on dividing the computation of the discrete Fourier transform (DFT) of a sequence into shorter DFTs and utilizing the symmetry and periodicity of $e^{-i2\pi kn/N}$. Two common classes of FFT algorithms are decimation in frequency (DIF) algorithms and decimation in time (DIT) algorithms. The simple DIF and DIT examples below assume an input sequence with length $N = 2^n$ for some $n \in \mathbb{N}$. Enhanced methods exist for dealing with arbitrary sequence lengths. A shorthand $W_N = e^{-i2\pi/N}$ is used in the examples. The details on how the algorithms shuffle one or another of the domain indices can be studied from *Discrete-Time Signal Processing* by Oppenheim and Schaffer [83].

The DIF algorithms recursively split the computation by dividing the input sequence into two halves from the middle, keeping the time domain samples in order but resulting in shuffled output frequency index after the computation:

$$\hat{x}_k = \sum_{n=0}^{N-1} x_n W_N^{nk} = \sum_{n=0}^{N/2-1} x_n W_N^{nk} + \sum_{n=N/2}^{N-1} x_n W_N^{nk}$$

Substituting $n = m + N/2$ on the right hand side yields

$$\begin{aligned} \hat{x}_k &= \sum_{n=0}^{N/2-1} x_n W_N^{nk} + \sum_{m=0}^{N/2-1} x_{m+N/2} W_N^{(m+N/2)k} \\ &= \sum_{n=0}^{N/2-1} x_n W_N^{nk} + \sum_{m=0}^{N/2-1} x_{m+N/2} W_N^{mk} W_N^{Nk/2}. \end{aligned}$$

$$W_N^{N/2} = e^{-i\pi} = -1 \implies W_N^{Nk/2} = (-1)^k \text{ and}$$

$$W_N^2 = e^{-i4\pi/N} = e^{-i2\pi/(N/2)} = W_{N/2} \implies W_N^{2nj} = W_{N/2}^{nj},$$

so writing $k = 2j$, $j \in \{0, 1, \dots, N/2-1\}$, for even frequency indices gives

$$\hat{x}_{2j} = \sum_{n=0}^{N/2-1} x_n W_N^{n2j} + \sum_{n=0}^{N/2-1} x_{n+N/2} W_N^{n2j} = \sum_{n=0}^{N/2-1} (x_n + x_{n+N/2}) W_{N/2}^{nj}$$

and similarly $k = 2j + 1$ for odd indices gives

$$\hat{x}_{2j+1} = \sum_{n=0}^{N/2-1} x_n W_N^{n(2j+1)} - \sum_{n=0}^{N/2-1} x_{n+N/2} W_N^{n(2j+1)} = \sum_{n=0}^{N/2-1} (x_n - x_{n+N/2}) W_N^n W_{N/2}^{nj}$$

for odd indices. A DIF algorithm will continue splitting the sequences until all the $\hat{x}_{\text{even}}$ and $\hat{x}_{\text{odd}}$ sums contain only a single value, thus avoiding multiple computations of the same terms that would be needed if the definition of DFT would be directly used.

The DIT algorithms recursively split the input sequence into smaller subsequences of even and odd indices shuffling the time index before the computation but keeping the output frequency index in order:

$$\begin{aligned}
 \hat{x}_k &= \sum_{n=0}^{N-1} x_n W_N^{nk} \\
 &= \sum_{n=0}^{N/2-1} x_{2n} W_N^{2nk} + \sum_{n=0}^{N/2-1} x_{2n+1} W_N^{(2n+1)k} \\
 &= \sum_{n=0}^{N/2-1} x_{2n} W_{N/2}^{nk} + W_N^k \sum_{n=0}^{N/2-1} x_{2n+1} W_{N/2}^{nk},
 \end{aligned}$$

The two new sequences are both split into two and so on, until the sums are single values. A DIT algorithm collects the whole DFT from the individual values after the splitting is complete, thus avoiding multiple computations of the same terms similarly to DIF algorithms.

The first known FFT algorithm was created by the German mathematician, astronomer, and physicist Carl Friedrich Gauss who wrote a DIF algorithm in 1805 while studying the orbits of asteroids Pallas and Juno. Gauss developed his method two years before Joseph Fourier, after whom the Fourier series and transforms are named, realized his work on functions as infinite harmonic series. [40]

3.3.2 Efficient computing of the wavelet transform

An algorithm's time complexity (running time) can be described as a function f of its input size $n \in \mathbb{N}$. Big O notation describes the upper bound of an algorithm's time complexity. $f(n) \in O(g(n))$ if a constant $c > 0$ exists so that $0 \leq f(n_0) \leq cg(n_0)$ for all $n \geq n_0$. In other words, the function $f(n)$ grows no faster than $g(n)$ for sufficiently large n , so $g(n)$ can be taken as the algorithm's worst case running time as a function of its input size. [22]

Computing the wavelet transform (Equation 3.10) using ψ_0 would require summing N computations for each of the N scales resulting in $O(N^2)$ time complexity. Using ψ^* and computing a discrete Fourier transform and an inverse discrete Fourier transform with a fast Fourier transform algorithm reduces the time complexity to $O(N \log_2 N)$ [96]. For example, let us consider the data used for this project. A very crude approximation of the training data set size N is about 10 million rows. Computing the wavelet transforms with the corresponding $O(N^2)$ time complexity algorithm would mean $(10^7)^2 = 10^{14}$ operations per column. The amount of operations is significantly reduced, to $10^7 \log_2(10^7)$ per column, by using a FFT algorithm.

$$\frac{10^{14}}{10^7 \log_2 10^7} = \frac{10^7}{\log_2 10^7} \approx 430043,$$

so if using a FFT algorithm would result in one minute of computation per column on a laptop, then the same computation with ψ_0 would take approximately 10 months per column with the same machine.

The inverse discrete Fourier transform is defined as

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} \hat{x}_k e^{i2\pi kn/N}, \quad (3.13)$$

where $n \in \{0, 1, \dots, N-1\}$ is the time index [95]. The Fourier transform of the scaled wavelet $\psi^*(t/s)$, normalized to unit energy at each scale, is written as $\hat{\psi}(s\omega) = \sqrt{\frac{2\pi s}{\delta t}} \hat{\psi}_0(s\omega)$, where $\hat{\psi}_0$ is the Fourier transform of the mother wavelet ψ_0 .

$W_n(s)$ (Equation 3.10) is a convolution of x_n and ψ^* and by the convolution theorem (Appendix A.3), the Fourier transform of a convolution equals a product of Fourier transforms of the convoluted sequences. In this case, the product is $\hat{x}_k \hat{\psi}^*(s\omega_k)$, the inverse discrete Fourier transform ($\hat{\psi}^*(s\omega_k)$ is a function of discrete angular frequencies) of which is

$$\frac{1}{N} \sum_{k=0}^{N-1} \hat{x}_k \hat{\psi}^*(s\omega_k) e^{i\omega_k n \delta t}, \quad (3.14)$$

where angular frequency is defined as

$$\omega_k = \begin{cases} \frac{2\pi k}{N\delta t}, & k \leq \frac{N}{2} \\ -\frac{2\pi k}{N\delta t}, & k > \frac{N}{2} \end{cases}.$$

By the convolution theorem, this inverse transform equals the inverse transform of the Fourier transform of $W_n(s)$. In other words, it equals $W_n(s)$. This equality allows the use of a fast Fourier transform algorithm and so the time complexity of the wavelet transform can be reduced from $O(N^2)$ to $O(N \log_2 N)$.

The inverse discrete Fourier transform produces a Fourier series. Fourier series expects cyclical functions, and consequently, a finite time series will result in errors at the beginning and at the end of the power spectrum. The cone of influence, where the errors are significant, is not of concern regarding this work because in the Pc5 frequency range it is small and both ends of the transformed data sequences are padded with additional data [68]. The results of the transforms are obtained by removing the added data from the beginning and from the end of the sequences after the wavelet transform has been applied.

4. Data

Data for training, validating, and testing the nowcasting models were acquired from Coordinated Data Analysis Web [19] and SuperMAG [98] websites.

4.1 SuperMAG

SuperMAG data repository is a worldwide collaboration of organizations and national agencies. It provides easy access to validated ground magnetic field perturbation data from multiple measurement stations. The focus of the repository is on the variations caused by electric currents in the ionosphere and the magnetosphere, which induce magnetic field perturbations observed on the ground. [98]

SuperMAG data is resampled to one minute or, when available, one second temporal resolution and all units are converted into nanoteslas. Manual and automated correction routines are used to remove artifacts. Data are rotated into local geomagnetic coordinates, and the Earth main field baseline is subtracted by an automated method as a last step. [32] [33]

Newell and Gjerloev presented partial SuperMAG ring current indices and the global SuperMAG ring current index (SMR) as an average of the partial indices in *SuperMAG-based partial ring current indices* [80]. The SMR index, which represents the ring current's intensity, uses SuperMAG data from 98 measurement stations between -50 and 50 degrees of geomagnetic latitude. The large number of stations allows a division into magnetic local-time (MLT) sectors SMR-00, SMR-06, SMR-12, and SMR-18. Each six hour sector is named according to its local-time center point. For example, SMR-00 refers to the index obtained from magnetometer stations located in the MLT sector ranging from 21:00 to 03:00. The sectors are also referred to as night (00), dawn (06), day (12), and dusk (18). The day sector refers to the sun-facing side of the magnetosphere. Global SMR index is defined as

$$SMR = \frac{SMR-00 + SMR-06 + SMR-12 + SMR-18}{4}.$$

Averaging of partial indices, as opposed to averaging of all stations globally, ensures that each sector will contribute 25% to the global SMR index. The five SMR indices

representing the intensity of the ring current in nanoteslas are available at SuperMAG data repository and they are nowcasted in this thesis. Disturbances of the ring current manifest in the indices as negative values, and the greater the disturbance, the smaller the value [23].

SuperMAG repository has also magnetic field values defined as $B = (B_N, B_E, B_Z)$, where B_N is local magnetic north, B_E is local magnetic east, and B_Z is vertically down [80]. An MLT dependent Pc5 ULF wave power index was developed in Lipsanen's 2024 master's thesis *Correlation of solar wind parameters and MLT dependent ground-based ULF wave index during large solar wind structures* by applying a scale averaged wavelet transform (Section 3.3) on the SuperMAG magnetic field horizontal component values

$$\sqrt{B_N^2 + B_E^2} = B_H \quad (4.1)$$

from stations located in the geomagnetic latitude range of 60 to 70 degrees [68]. The transform was computed with PyCWT [62], a Python library for continuous wavelet spectral analysis.

Table 4.1 lists the five aggregate Pc5 indices based on magnetic local time that are nowcasted in this thesis.

Sector	MLT	Index
Global	all times	Pc5 _{global}
Dawn	3-9 MLT	Pc5 _{dawn}
Day	9-15 MLT	Pc5 _{day}
Dusk	15-21 MLT	Pc5 _{dusk}
Night	21-03 MLT	Pc5 _{night}

Table 4.1: Pc5 ULF wave power index sectors.

The Pc5 ULF wave power of an MLT sector is the average Pc5 index value from all the stations of that sector and Pc5_{global} is the average of all the sectors so the Pc5 indices are constructed in the same way as the SMR indices.

4.2 Coordinated Data-Analysis Web

National Aeronautics and Space Administration (NASA) maintains the Coordinated Data-Analysis Web repository which hosts data from current and past missions and projects. Many datasets from the current missions are updated regularly, and also past data are reprocessed [19]. Advanced Composite Explorer (ACE) and Wind are two NASA spacecraft on halo orbits around the Sun-Earth L1 Lagrange point [76] [77], which is located inside Earth's orbit about 1.5 million kilometers from Earth towards the Sun [25].

The Lagrange points are five positions in which the gravitational pull of two masses equals the centripetal force required for a small object moving with the masses. They were introduced by the Italian-French mathematician Joseph-Louis Lagrange in his 1772 paper *Essai sur le Problème des Trois Corps* [64]. The Lagrange points can be used to place spacecraft in space for remaining in a fixed position with minimal fuel consumption [78]. A halo orbit is a circular or elliptical loop around a Lagrangian point [1], whereas a Lissajous orbit, named after French mathematician Jules Antoine Lissajous (1822–1880), is a more complex natural curve [2].

NASA launched the ACE spacecraft from Florida’s Cape Canaveral on the 25th of August 1997. It was declared operational on the 21st of January 1998 after reaching its intended halo orbit around L1 Lagrange point. The main purpose of the spacecraft, which is still operational excluding one instrument, is to investigate matter ejected from the Sun. [76]

Wind was launched from the same location on the 1st of November 1994 to the same destination. Its primary purpose was to observe solar wind about to impact the magnetosphere of the Earth. Due to two other missions, ACE and SOHO (a collaboration of NASA and the European Space Agency [79]), its route was changed and it circulated Earth until November 1996, when it entered a halo orbit around the L1 point. On the 18th of November 1998 Wind was taken out of the ecliptic plane, up to 60 degrees below and above it in order to take measurements from previously unstudied interplanetary space and magnetosphere. It entered a Lissajous orbit around L1 in 2004 and in 2020 it returned to a halo orbit around L1. Wind is also still operational. [77]

4.3 Data processing

PySPEDAS is a Python implementation of Space Physics Environment Data Analysis Software framework [46]. It was used to access ACE and Wind mission data from the CDAWeb for this project. SuperMAG offers an application programming interface [101] for data loading but it was not suitable for downloading large amounts of data at the time of working on this thesis so it was not used.

The SMR index and the partial SMR indices were manually loaded from SuperMAG [99] as yearly one minute time resolution csv-files containing all the five indices. The files were loaded into Polars dataframes with a Python script one by one. The datetime column was processed into a suitable form for Polars and CDAWeb data date format, and any rows with fill values were removed.

The data for computing and nowcasting the Pc5 ULF wave power index was downloaded manually from SuperMAG [100] as yearly one minute time resolution NetCDF-files. In addition to datetime, the data included three-dimensional magnetic field perturbation

values in nanoteslas, magnetic local time, three letter station identifier, geographic latitude and longitude and magnetic colatitude. The yearly files were loaded into Polars dataframes one file at a time and gaps longer than five minutes were searched. No such gaps were found, which was not surprising considering that for each timestamp there were about 50 stations in the latitude range of 60 to 70 degrees. Next step was to compute the horizontal magnetic perturbation B_H (Equation 4.1) from the east- and north perturbation values and add it to the dataframe. After adding the B_H value, Pc5 indices were computed following the Helsinki University Space Physics Group code: PyCWT Python package [62] was used for the wavelet transform with the parameters listed in Table 4.2.

Table 4.2: PyCWT wavelet transform parameters.

Parameter	Value / Definition
Sampling interval of the data (dt)	60 seconds
Smallest resolvable scale of the wavelet (s_0)	$2dt$
Spacing between discrete scales (dj)	$\frac{1}{12}$
Number of scales (J)	$\frac{9}{dj}$

These parameters, and PyCWT Morlet mother wavelet function with $\omega_0 = 6$, were given to the PyCWT continuous wavelet transform function and the result was averaged by scales as described in Section 3.3.

Yearly ACE and Wind data from the CDAWeb were loaded with PySPEDAS. Three dimensional vector measurements of interest to this project were the spacecraft position, magnetic field components in the geocentric solar magnetosphere (GSM) system, and solar wind GSM velocity. Selected scalar measurements were proton density, thermal speed (from which temperature was derived), and magnetic field magnitude. Initially all the available data from 1.1.1999 to 31.12.2023 was downloaded and afterwards a function for loading the data from disk instead of downloading from the CDAWeb was created and used for loading the data into Polars dataframes one year at a time. Values considered unphysical were removed using the limits presented in Table 4.3 and missing values were interpolated following Lipsanen [68].

Table 4.3: CDAWeb parameters' limits.

Parameter	Minimum	Maximum
Magnetic field magnitude	0 nT	100 nT
Magnetic field x-component	-100 nT	100 nT
Magnetic field y-component	-100 nT	100 nT
Magnetic field z-component	-100 nT	100 nT
Proton density	0 cm^{-3}	100 cm^{-3}
Temperature	$1 \times 10^3 K$	$1 \times 10^8 K$
Velocity x-component	-1500 km/s	1500 km/s
Velocity y-component	-1000 km/s	1000 km/s
Velocity z-component	-1000 km/s	1000 km/s
Spacecraft position x	200 R_e	300 R_e
Spacecraft position y	-100 R_e	100 R_e
Spacecraft position z	-100 R_e	100 R_e

A simple physical solar wind propagation model, which gives the propagation time in seconds, was used in this project. It is defined as

$$\Delta t = \frac{||\vec{r}||^2 R_E}{|\vec{r} \cdot \vec{v}|} = \frac{(x^2 + y^2 + z^2) R_E}{|xv_x + yv_y + zv_z|}, \quad (4.2)$$

where $\vec{r} = (x, y, z)$ is the spacecraft position in Earth radii, $\vec{v} = (v_x, v_y, v_z)$ is the solar wind velocity in km/s, and R_E is the Earth radius 6371 km. The datetime of each set of measurements was shifted based on the delay time computation. CDAWeb data contained denser than one minute data so after the timeshift it was resampled to one minute intervals to match SuperMAG time resolution. The resampling was done by averaging the values within each one minute interval.

After processing both SuperMAG and CDAWeb dataframes of a given year, the two dataframes were merged into one by adding the values with the nearest datetime from the CDAWeb dataframe to new columns in the SuperMAG dataframe. The merging operation used a one minute filter so the values added to each row of the SuperMAG frame from the CDAWeb frame were within one minute range of the SuperMAG values of the row. After joining the values, lagged values of some features and all the targets were added to the final combined dataframe and it was saved to disk. After all the yearly data were processed, they were combined into two new files containing all the data from 1.1.1999 to 31.12.2023. One file for the SMR index prediction and one file for the Pc5 index prediction. 2024 data was processed similarly for the purpose of testing the models after training and validating with the 1999-2023 data.

5. Method implementation

Implementation of the XGBoost models for nowcasting the SMR and ULF Pc5 indices is described in this chapter as well as feature engineering and model evaluation process. A Python script was developed with DMLC XGBoost Python package [43] for the nowcasting tasks. At the initial stage one multi-output XGBoost model which predicts several targets instead of one, was used for nowcasting SMR indices for the present time. After the pipeline for loading the preprocessed data, training the model, making predictions, and evaluating the results was functional, the multi-output model was replaced with five single-output models. Similarly, for nowcasting Pc5 indices, one model per target was used so that the hyperparameters could be tuned for each target individually. At the late stages of script refinement the nowcasting target was changed from the present to three hours in the future, so the target values were shifted three hours into the past and resulting rows with no target values were removed. All the models used weighted errors for three two week periods containing the largest number of extreme values. Target values in the 0.001 and 0.999 quantiles were defined to be extreme and all the 15 periods were chosen from different years. During training, the nowcasting errors ε_i in the extreme values periods counted as $5\varepsilon_i$, directing the models to pay more attention to the extreme conditions.

Polars is an open-source library for data manipulation [45]. The benchmarks from May 2025 show two orders of magnitude faster performance compared to Python data analysis library Pandas [47], which is very commonly used in machine learning implementations. Pandas was not efficient enough for data processing and model training on a personal GPU laptop. With Polars and a yearly approach to some of the data preprocessing as described in Section 4.3, there was no need for high-performance computing resources.

5.1 Feature engineering

The process of transforming data into a data set that can be used by a machine learning model is called feature engineering [17]. In addition to preparing the data as described in Section 4.3, further experiments were made with the features used in training the

models. Initially all the columns from the loaded data were used, and at later stages additional versions of both SMR and Pc5 nowcasting training data were created by removing features from the original datasets. Features were dropped based on XGBoost’s internal feature importance score. XGBoost has three options for computing the importance scores: weight, gain, and cover. Gain-based scoring was used in this work. It assigns a score for each feature based on the error reduction from each split that uses the feature. Gain-based scores sum to one. Weight-based importance counts the times each feature is used in splits and cover-based method computes the number of samples each feature affects. New features were created based on the existing ones or on time. Experiments with ratios, products, sums, squares, and combinations of the aforementioned operations on the top two features based on the feature importance score were conducted. All of them except Unix time, which stands for seconds since 1.1.1970 00:00:00 UTC, were found to be meaningless for the models’ performance. The whole CDAWeb data was discarded for the models nowcasting the Pc5 indices: XGBoost could not improve its performance by training with it compared to only using lagged Pc5 indices’ values within the time and resource limits of this thesis project. SMR models did not benefit from proton density or temperature values so they were not used. Also magnetic field x- and y-components were found to be useless for the models’ predictions so only the z-component was included. The y-component was the only one used of the three velocity components as x- and z-components did not contribute to the prediction performance sufficiently.

5.2 XGBoost hyperparameters

DMLC XGBoost package [43] functions offer many parameters for tuning model performance. The parameters and their values for this project were selected through an iterative experimental process of running the script with different combinations of parameters and parameter values and they are explained in this section. Automated tools for hyperparameter tuning exist, for example Optuna [5] [44], which was used for obtaining the final parameters. All of the work was done on a personal laptop so the amount of computation was limited. The hyperparameter values and Optuna functions are part of the script available at GitHub [60].

The hyperparameters adjusted for XGBoost models were ‘seed’, ‘device’, ‘max_depth’, ‘min_child_weight’, ‘learning_rate’, ‘lambda’, ‘alpha’, ‘gamma’, ‘subsample’, ‘colsample_by_tree’, and ‘sampling_method’. In addition to these hyperparameters, the DMLC XGBoost package allows setting the maximum number of trees used with ‘num_boost_rounds’ and stopping the algorithm earlier than initially defined with ‘early_stopping_rounds’ if the results are not improving at least every ‘early_stopping_rounds’ iteration.

Device parameter was set to 'cuda' so that the algorithm would run on GPU instead of CPU. The random seed parameter 'seed' controlling the reproducibility of results involving randomness was explicitly set to 1 for clarity. Maximum depth of regression trees is given by a self-explanatory 'max_depth' parameter. If there was a leaf with a sum of weights less than 'min_child_weight' after a tree partition step, then the split into new leaves was not made. Larger 'min_child_weight' values drive the algorithm to build shallower trees because each split must result in leaves having samples with a larger combined weight. Sum of weights in a regression task, such as the ones in this thesis, is simply the number of samples in the leaf.

Learning rate $\eta \in (0, 1]$ is a scaling factor for the new tree $f_t(x_i)$ at the t -th step and i -th sample (Equation 3.3). The smaller it is, the smaller the adjustment $\eta f_t(x_i)$ to the predicted value. Tuning η down means more gradual and slower learning, as the effect of a single tree decreases. In practice, smaller η means that more trees are going to be used as the loss between real and predicted value does not decrease as fast as with a larger η .

Lambda and gamma are the λ and γ parameters in equations 3.2 and 3.8. Lambda scales L2 regularization (Subsection 2.1.2) and gamma is the minimum loss reduction needed for making a new split. Alpha scales L1 regularization and defaults to zero. It is not shown in Equation 3.2. The regularization term

$$\Omega(f_k) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

becomes

$$\Omega(f_k) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 + \alpha \sum_{j=1}^T |w_j|$$

for $\alpha \neq 0$.

The 'subsample' parameter scales the proportion of training samples used by each tree. Default value 1 means using all samples, other values between 0 and 1 mean that the algorithm samples that proportion randomly at the beginning of each iteration. Similarly, the proportion of used columns (features) is controlled by the 'colsample_by_tree' parameter. For example, setting 'subsample' to 0.6 and 'colsample_by_tree' to 0.9 results in 60% of the data and 90% of features being used in each iteration as an attempt to reduce possible overfitting.

Finally, 'gradient_based' was selected for 'sampling_method' parameter to optimize GPU-usage for speed. With this setting the algorithm uses $\sqrt{g^2 + \lambda h^2}$, where g is the gradient and h is the Hessian of the sample, to determine the probability of using the sample for the current iteration f_t when the 'subsample' parameter value is below 1. Thus it prioritizes the samples contributing more to loss reduction, potentially improving the model performance.

5.3 Model performance evaluation

Model performance evaluation was conducted through time series cross-validation on 1999-2023 data. Considering the short time horizon of nowcasting three hours into the future, validation set length was chosen to be two weeks. Three two week sets for each target were chosen from the two week periods containing the largest amount of extreme values, defined as in the opening paragraph of this Chapter 5. In addition, a two week period from the very end of each year were chosen from the remaining 10 years. This resulted in using 50 weeks, which is approximately 3.7% of the whole 1999 – 2024 data, for validation.

Squared error was chosen for the objective function for all models as an attempt to account for the extreme values in both of the nowcasted indices, as it penalizes large errors more than small errors. Minimizing mean squared error is equivalent to minimizing root mean squared error, so RMSE was chosen as the evaluation metric for all the models. It was used by the models after every iteration to check for early stopping.

The models were trained with data up to the first validation set and then nowcasted using the validation set data. After the first validation, models were trained with data up to the second validation set and so on, as described in Subsection 2.1.3. The errors of each iteration were saved and hyperparameters were tuned based on the average performance over all the validation sets.

6. Results

The results presented in this chapter were obtained with the test data set consisting of the entire year 2024 year, which represents approximately 3.8% of the total data. As discussed in Section 5.1, models nowcasting Pc5 indices did not find any parameters selected from the CDAWeb data useful in terms of enhancing the prediction performance, thus preventing any interpretation on the relation between solar wind parameters and the index values. On the other hand, SMR models did find most of the CDAWeb data variables somewhat useful even though the lagged values of the targets clearly dominated the importance score lists. The importance of the CDAWeb features in predicting SMR indices is discussed here in addition to the nowcasting results.

6.1 SMR indices

The true and nowcasted values of the 2024 global SMR index are shown in Figure 6.1 as two time series: a blue line for the true values and an orange line for the nowcasted values. The XGBoost model managed to capture the patterns very well, except for the outlier values representing either particularly strong ring current (highly negative SMR) or strongly compressed magnetosphere (highly positive SMR). More extreme true values resulted in larger nowcasting error. The models nowcasting the partial SMR indices performed similarly but they were not as accurate. The results are visualized in Figure ??.

Table 6.1 lists the error (Appendix B) values of the SMR models. A very small median absolute error of only 0.2 nT, a small mean absolute error of 0.83 nT and a larger root mean squared error of 6.28 nT confirm the visual interpretation of a good baseline performance combined with a significantly weaker capacity for nowcasting outlier values.

It is likely that the outliers, some of which are extreme, push also the scaled mean errors up. The fluctuations of SMR indices might be mostly smooth enough for a naive forecaster, which always predicts the previous value, to perform adequately, but contain sufficiently large outliers for the XGBoost models to lose track. The models predict the mean of a leaf’s training data values for any new observation falling into the leaf. In case of extreme values, the average can be significantly smaller or larger than the actual

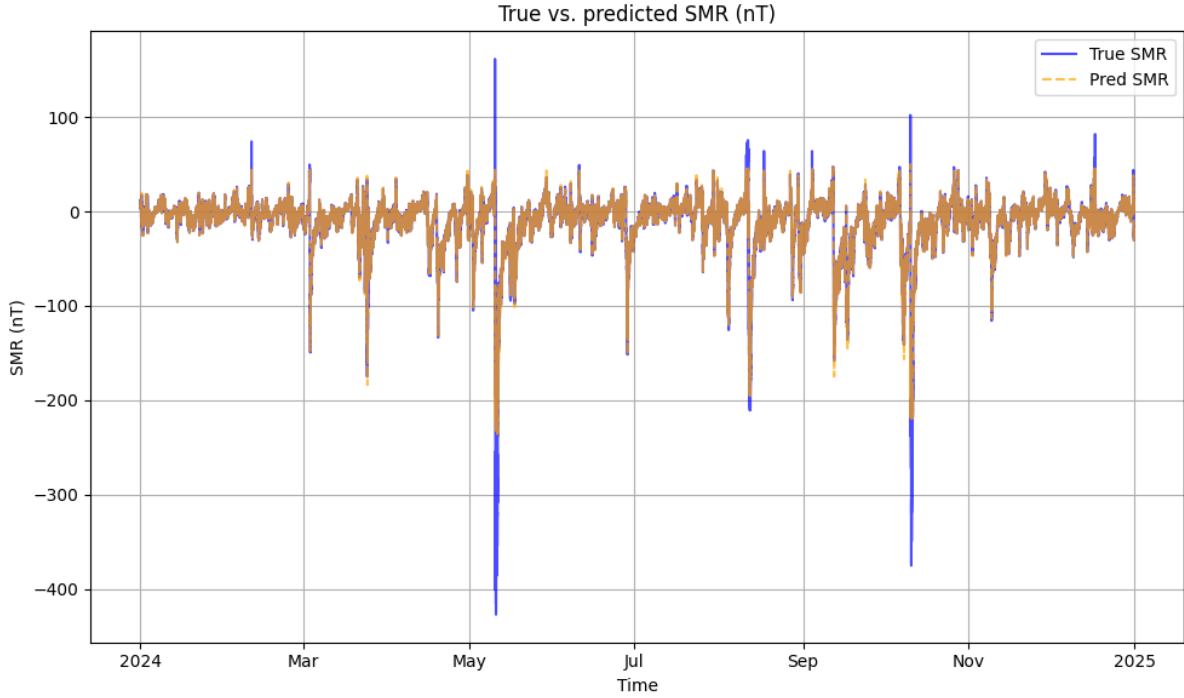


Figure 6.1: Global SMR index nowcast for 2024.

value. The time series of the results display this kind of XGBoost clipping behavior. On the other hand, the naive forecaster can climb up and down the extreme values with very small errors if the fluctuations are smooth enough. The naive forecaster is a very strong model for the SMR indices given the short time horizon of three hours used in the nowcast.

Table 6.1: SMR model performance metrics.

Metric	Global SMR	SMR00	SMR06	SMR12	SMR18
MdAE	0.20 nT	0.22 nT	0.19 nT	0.30 nT	0.29 nT
MAE	0.86 nT	1.04 nT	1.03 nT	1.13 nT	1.01 nT
RMSE	6.55 nT	8.21 nT	8.65 nT	7.50 nT	6.65 nT
MdASE	0.93	0.67	0.76	0.80	0.83
MASE	3.95	3.22	4.03	3.02	2.94
RMSSE	15.22	12.76	16.80	10.59	9.93

6.2 SMR models' feature importance

XGBoost's internal gain feature importance was used in pruning the models' features as discussed in Section 5.1. In addition, Shapley Additive Explanations (SHAP) summary [69] [70] was used as an attempt to gain further insight into the relations between features

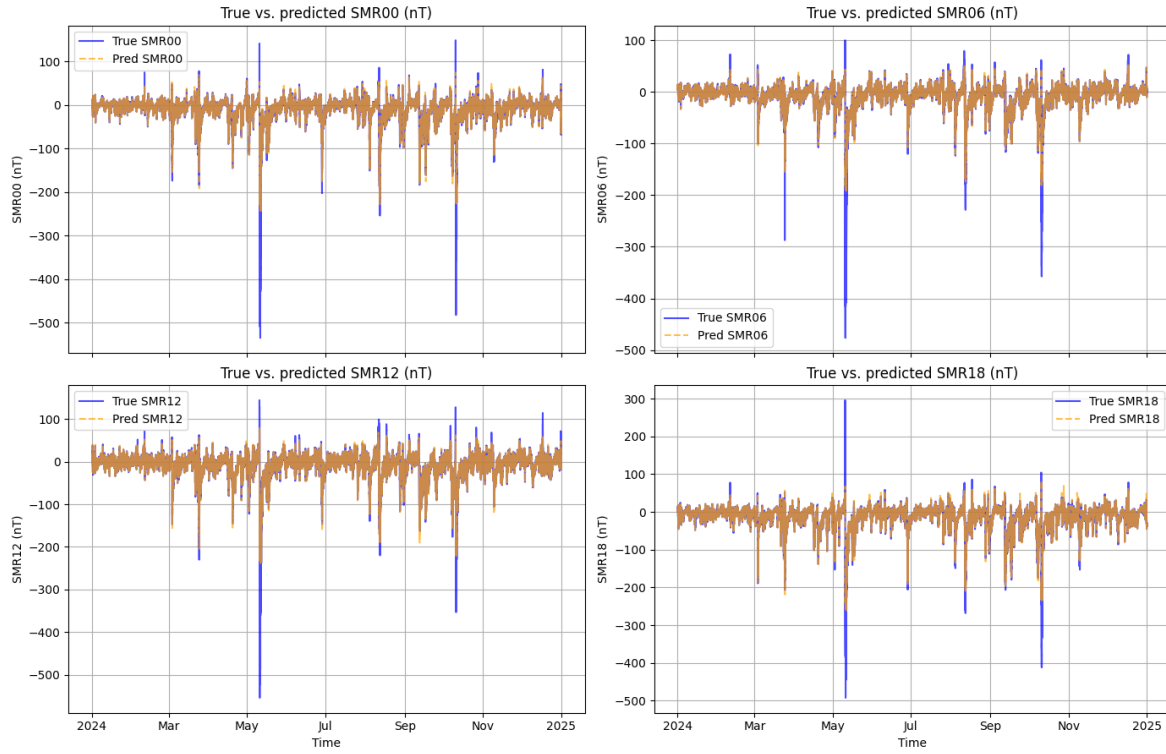


Figure 6.2: Partial SMR indices nowcast for 2024.

and each predicted value, as well as to their importance. A SHAP summary applied to the models did not provide additional benefit in choosing the features as the amount of information was overwhelming: summary shows each individual prediction. Analyzing the effect of each feature to the predictions and each other in greater depth would have been a project not fitting the scope of this thesis, as there are hundreds of thousands of predictions for the year 2024. The summary plot containing all of the predictions is not suitable for any meaningful analysis, as the number of dots is so large (one for each feature for all predictions) that it is impossible to get an accurate view on the effects. An example summary plot and a short non-technical description of SHAP can be found from Appendix D.2 and further discussion on SHAP from Chapter 7.

Table 6.2 lists the four most useful CDAWeb features and their importance scores per model. Lagging has been ignored in order to produce a compact summary, so for example the importance score for the x -coordinate of spacecraft position could taken from a four minutes lag for some model, two minutes for some other, and a third model's score could be taken from the variable without any lagging. In addition, the models' importances have several different lagged values of the variable among the top scoring ones but only the highest value is included in the summary table. The importances are visualized in Figure 6.3. The complete lists of the 43 features used by the SMR models and their scores are in Appendix D.1. Magnetic field magnitude, spacecraft position and the z -component

of the magnetic field were the most important CDAWeb features overall according to the models' importance scores. Exceptions were unix time reaching the third position with a score of 0.0097 for the SMR-06 model and thermal speed as the fourth most important feature for the SMR-00 model with a score of 0.0072 while the y-component of spacecraft position was the third most important with a score of 0.0073.

Table 6.2: Four overall most useful CDAWeb features and their scores in SMR models.

Feature	SMR-00	SMR-06	SMR-12	SMR-18	SMR
Magnetic field magnitude	0.0182	0.0107	0.0129	0.0130	0.0169
Spacecraft position x-coordinate	0.0159	0.0098	0.0053	0.0038	0.0050
Magnetic field z-component	0.0067	0.0048	0.0060	0.0054	0.0068
Spacecraft position y-coordinate	0.0073	0.0065	0.0079	0.0045	0.0055

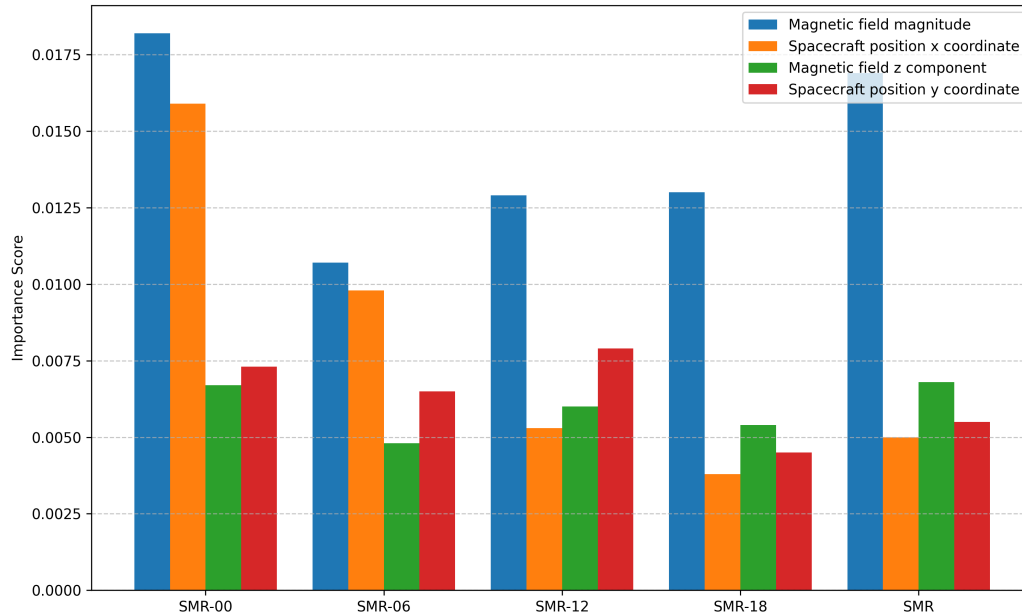


Figure 6.3: The four most useful CDAWeb features and their importance scores.

Some of the selected CDAWeb features have extremely low importance scores and the global SMR model declares approximately 92% of its performance being dependent on the lagged values. SMR-18 is not much behind with its approximately 90% dependence on the lags. The remaining approximate importance attributed to the CDAWeb features in case of the other partial index models is higher: 18%, 13%, and 15% for SMR-00, SMR-06, and SMR-12. Thus the usefulness of the selected features for the models cannot be ignored, even though many of the values might look insignificant when considering them individually.

6.3 Pc5 indices

Pc5 index nowcasting models seem to deviate more than the SMR index models from the true values based on the time series graphs in Figures ?? and ??. Error values in Table 6.3 confirm this observation: the values are higher also when accounting for the larger range of the true values. The scaled errors are significantly smaller for the Pc5 index models and all below one, indicating that a naive forecaster for the Pc5 indices would not perform as well as for the SMR indices. The target values are most likely characterized by large and rapid changes, which explains why a naive forecaster would struggle.

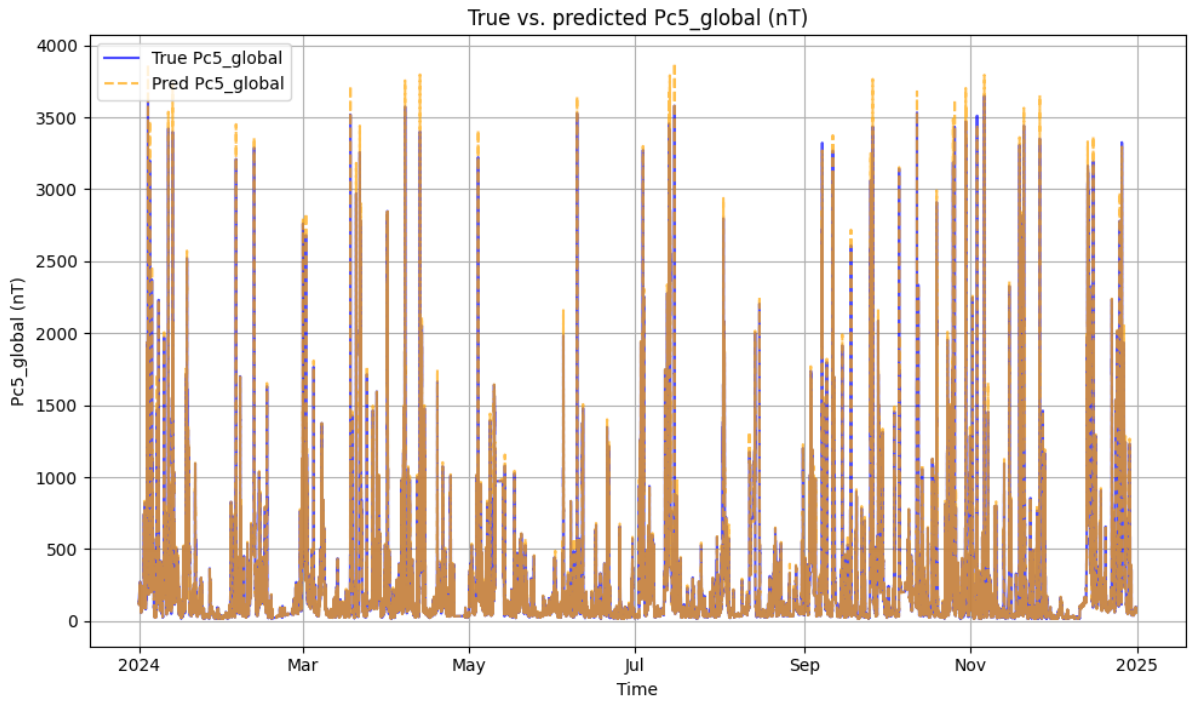


Figure 6.4: Global Pc5 index nowcast for 2024.

Table 6.3: Pc5 model performance metrics.

Metric	Pc5_global	Pc5_Night	Pc5_Dawn	Pc5_Day	Pc5_Dusk
MdAE	1.21 nT	17.87 nT	19.12 nT	19.97 nT	17.38 nT
MAE	3.80 nT	55.26 nT	56.19 nT	61.08 nT	54.53 nT
RMSE	16.17 nT	135.34 nT	132.03 nT	141.22 nT	128.52 nT
MdASE	0.12	0.07	0.08	0.09	0.08
MASE	0.37	0.23	0.24	0.26	0.25
RMSSE	0.41	0.22	0.22	0.25	0.23

Table 6.3 displays median absolute error MdAE, mean absolute error MAE, and

root mean squared error RMSE of the Pc5 models. Figure ?? shows that the partial Pc5 index values were more challenging for XGBoost nowcast than the global Pc5 index values in Figure ?? which the dedicated model predicted well.

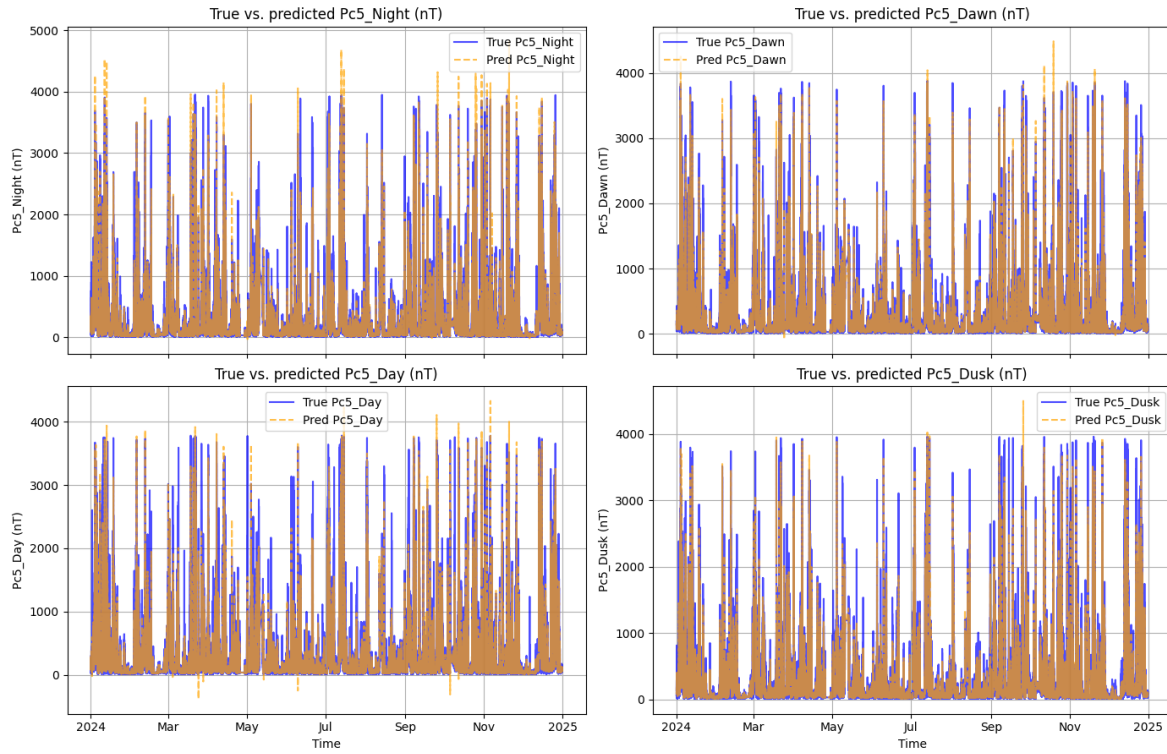


Figure 6.5: Partial Pc5 indices nowcast for 2024.

7. Discussion and future research

The process of hyperparameter tuning, feature engineering, and programming the implementation proved to be a very time-consuming and complex endeavor. Due to time constraints, the initial plan of comparing deep neural networks (Appendix C) and the XGBoost models was not realized as there was no time to implement neural networks after building the XGBoost models and evaluating them. The XGBoost models performed well but the results could most likely be improved with additional research, experiments, and high-performance computing resources. In retrospect, using high-performance computing, even though data processing and model training could be completed with a personal laptop, would have been beneficial for running more experiments, feature engineering, and hyperparameter tuning for possibly improving the overall result.

If the system would potentially have unrealized performance capabilities after further feature engineering and an exhaustive hyperparameter search, possibly with the help of high-performance computing resources, a classifier model could be trained. The classifier could be used for identifying shock events and also other types of important periods from the CDAWeb data, allowing the training of specialized models for different types of intervals and potentially increasing the nowcasting performance. For example, a classifier could flag certain periods in the data as geomagnetic storms and for those periods models specifically trained with storm-time data could be used. If a classifier would improve the results, then it should be investigated whether one classifier would be reliable enough for multiple event types, or would distinct classifiers be needed for each type of event. In addition, a model predicting the solar wind propagation delay from the data could be trained as an attempt to improve the simple vector delay used in this work.

Predicting single exact values can create a false sense of certainty. Uncertainty could be incorporated into the predictions of the system by computing 95% confidence intervals or some other measure of uncertainty and predicting, for example, $\hat{y}_0 = -45.2 \pm 2.3$ nT instead of $\hat{y}_0 = -45.2$ nT. The chosen metric for uncertainty could offer insights into the model and the underlying physical phenomena, while presenting a more realistic and informative view.

It would be interesting to develop deep neural networks and see if they would outperform an updated XGBoost ensemble. Additional experiments with different boosting

architectures could be performed, too. For example with LightGBM [37] which grows the trees differently compared to XGBoost and is usually faster to train. Deep learning could also be utilized to search for the relations between the solar wind measurements at the L1 point and the target indices. XGBoost did not perform well without lagged values of the targets and the SMR results rely mostly on the previous values of the indices, while the Pc5 models did not use CDAWeb data at all. Well designed deep neural networks might be able to capture meaningful patterns from the solar wind data alone for both nowcasting tasks. The difference in required computing resources could be recorded when developing new solutions providing an overview on that aspect of nowcasting the target indices.

The last - but not the least - topic for future research is a comprehensive SHAP analysis of the used models. In addition to feature summarizing, of which an example is provided in Appendix D.2, SHAP makes it possible to investigate interactions between features, 'memory' of the magnetosphere in a sense of comparing the effect of a feature for example 30 minutes ago, one hour ago, and at the current moment. It is also possible to investigate thresholds. For example, if a feature is linearly affecting predictions or has a threshold value, after which it becomes non-linear or stops affecting predictions. Completing a comprehensive SHAP analysis in this context requires a significant amount of work and only a short description with a summary example was included in this thesis.

Newell's coupling function

$$\frac{d\Phi_{MP}}{dt} = v^{4/3} B_T^{2/3} \sin^{8/3}(\theta_c/2) \quad (7.1)$$

represents the amount of energy transferred from the solar wind into the Earth's magnetosphere [81]. $B_T = \sqrt{B_y^2 + B_z^2}$ and $\theta = \arctan(B_y/B_z)$ in Equation 7.1 so the z-component of the magnetic field featuring high in the CDAWeb feature importances could indicate that the models captured a relation partly resembling the one described by Newell. However, the models did not utilize velocity except for the y-component and any correlation between Newell and feature importance is a hypothesis at best. Feature importance does not present any guaranteed relations between the satellite data and the target values considering physics, as it scores the features solely based on their effect on the models' predictive performance. Thus they do not necessarily tell anything about the underlying physics. In this case feature importance did not seem to capture the most important physical parameters well. Interestingly, the models found thermal speed, which could be a proxy for stormlike, or other exceptional periods, to be useful in addition to Unix time and satellite position.

While thermal speed is not generally considered to be among the most important solar wind parameters to influence geomagnetic activity, the large-scale solar wind structures driving high SMR activity could indirectly cause this finding. Geoeffective structures

are often compressed and high-speed streams have higher density plasma than the slow wind. [59]

Unix time could hint at the seasonality of solar wind (Section 2.3). The training data spans 25 years so it most likely includes two complete cycles of approximately 11 years. Satellite position could also point towards time as a useful feature, as the satellites have fixed orbits so the position could act as a shorter time cycle. Investigations into this could be performed with different kind of models, for example neural networks built specifically for catching the cyclic nature of solar wind. Combining them with SHAP analysis might yield interesting results.

Bibliography

- [1] Oxford University Press: Oxford Reference. <https://www.oxfordreference.com/display/10.1093/oi/authority.20110803095917502>. Accessed on the 17th of July 2025.
- [2] Oxford University Press: Oxford Reference. <https://www.oxfordreference.com/display/10.1093/oi/authority.20110803100108651>. Accessed on the 17th of July 2025.
- [3] C. Aggarwal et al. On the surprising behavior of distance metrics in high dimensional space. *Database Theory - ICDT'01*, pages 420–434, 2001.
- [4] L. Ahlfors. *Complex Analysis, 3rd edition*. McGraw-Hill Book Company, 1325 Avenue of the Americas, New York, NY 10019, USA, 1979.
- [5] T. Akiba et al. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. Association for Computing Machinery, 1601 Broadway, 10th Floor, New York, NY 10019-7434, USA, 1999.
- [6] T. Apostol. *Calculus, Volume 1, Second Edition*. Wiley, 111 River Street, Hoboken, NJ 07030, USA, 1975.
- [7] D. Baker. An extreme distortion of the Van Allen belt arising from the ‘Hallowe’en’ solar storm in 2003. *Nature*, 432:878–881, 2004.
- [8] R. Bellman. *Adaptive Control Processes: A Guided Tour*. Princeton University Press, 41 William Street Princeton, New Jersey 08540, 1961.
- [9] C. Bentéjac, A. Csörgő, and G. Martínez-Muñoz. A comparative analysis of gradient boosting algorithms. *Artificial Intelligence Review*, 54(8):1937–1967, 2021.
- [10] K. Beyer et al. When is ‘nearest neighbor’ meaningful? *Database Theory - ICDT'99*, pages 217–235, 1999.
- [11] M. Bonavita. On some limitations of current machine learning weather prediction models. *Geophysical Research Letters*, 51, 2024.

- [12] J. Borovsky. Is our understanding of solar-wind/magnetosphere coupling satisfactory? *Frontiers in Astronomy and Space Sciences*, 8, 2021.
- [13] J. Bortnik and E. Camporeale. Ten ways to apply machine learning in earth and space sciences. *Eos*, 102, 2021.
- [14] L. Breiman. Arcing the edge. *Annals of Probability*, 26:1683–1702, 1998.
- [15] L. Breiman. Prediction games and arcing algorithms. *Neural Computation*, 11(7):1493–1517, 1999.
- [16] L. Breiman et al. *Classification and Regression Trees*. Chapman & Hall, 6000 Broken Sound Parkway NW, Suite 300 Boca Raton, Florida 33487-2742, USA, 1984.
- [17] A. Burkov. *The Hundred-page Machine Learning Book*. <https://github.com/Chrisackerman1/The-Hundred-Page-Machine-Learning-Book>. Accessed on the 22th of July 2025.
- [18] E. Camporeale. The challenge of machine learning in space weather: Nowcasting and forecasting. *Space Weather*, 17:1166–1207, 2019.
- [19] CDAWeb. <https://cdaweb.gsfc.nasa.gov/>. Accessed on the 1st of July 2025.
- [20] S. Chapman and V. Ferraro. A new theory of magnetic storms. *Terrestrial Magnetism and Atmospheric Electricity*, 36(3):77–97, 1931.
- [21] T. Chen and C. Guestrin. Xgboost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [22] T. Cormen et al. *Introduction to Algorithms Fourth Edition*. The MIT Press, 255 Main Street, 9th Floor, Cambridge, Massachusetts 02142, USA, 2022.
- [23] I. Daglis et al. The terrestrial ring current: Origin, formation, and decay. *Reviews of Geophysics*, 37:407–438, 1999.
- [24] DMLC XGBoost. <https://xgboost.readthedocs.io/en/stable/tutorials/model.html#additive-training>. Accessed on the 13th of August 2025.
- [25] European Space Agency. https://www.esa.int/Science_Exploration/Space_Science/L1_the_first_Lagrangian_Point. Accessed on the 13th of August 2025.
- [26] M. Farge. Wavelet transforms and their applications to turbulence. *Annual Review of Fluid Mechanics*, 24(1):395–458, 1992.

- [27] R. Fitzpatrick. <https://farside.ph.utexas.edu/teaching/plasma/lectures1/node25.html>. Accessed on the 20th of September 2025.
- [28] G. Folland. *Fourier Analysis And Its Applications*. Brooks/Cole, Belmont, California, 94002, USA, 1992.
- [29] Y. Freund and R. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.
- [30] J. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.
- [31] J. Friedman. Stochastic gradient boosting. *Computational Statistics & Data Analysis*, 38(4):367–378, 2002.
- [32] J. Gjerloev. A global ground-based magnetometer initiative. *Eos*, 90(27):230–231, 2009.
- [33] J. Gjerloev. The supermag data processing technique. *Journal of Geophysical Research*, 117(A09213), 2012.
- [34] W. Gonzales et al. What is a geomagnetic storm? *Journal of Geophysical Research: Space Physics*, 99:5771–5792, 1994.
- [35] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. The MIT Press, 255 Main Street, 9th Floor, Cambridge, Massachusetts 02142, USA, 2016.
- [36] K. Grąbczewski. *Meta-Learning in Decision Tree Induction*. Springer, Gewerbe-strasse 11, Cham, Ch 6330, Switzerland, 2014.
- [37] K. Guolin et al. LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30(4):3146–3154, 2017.
- [38] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd edition. Springer, Gewerbstrasse 11, Cham, Ch 6330, Switzerland, 2009.
- [39] D. Hathaway. The solar cycle. *Living Reviews in Solar Physics*, 12, 2015.
- [40] M. Heideman, D. Johnson, and C. Burrus. Gauss and the history of the fast fourier transform. *IEEE ASSP Magazine*, 1(4):14–21, 1984.
- [41] Helsinki University Space Physics Group. <https://blogs.helsinki.fi/spacephysics/>. Accessed on the 22nd of January 2026.

- [42] A. Hoerl and R. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.
- [43] <https://github.com/dmlc/xgboost>. <https://xgboost.readthedocs.io/en/latest/>. Accessed on the 4th of July 2025.
- [44] <https://github.com/optuna/optuna>. <https://optuna.org/>. Accessed on the 3th of August 2025.
- [45] <https://github.com/pola-rs/polars>. <https://docs.pola.rs/>. Accessed on the 18th of July 2025.
- [46] <https://github.com/spedas/pyspedas>. <https://pyspedas.readthedocs.io/en/latest/>. Accessed on the 15th of July 2025.
- [47] <https://pola.rs/>. <https://pola.rs/posts/benchmarks/>. Accessed on the 18th of July 2025.
- [48] G. Hughes. On the mean accuracy of statistical pattern recognizers. *IEEE Transactions on Information Theory*, 14(1):55–63, 1968.
- [49] E. Huttunen et al. Asymmetric development of magnetospheric storms during magnetic clouds and sheath regions. *Geophysical Research Letters*, 33(6):4, 2006.
- [50] R. Hyndman and A. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22:679–688, 2006.
- [51] International Business Machines Corporation. <https://www.ibm.com/think/topics/supervised-vs-unsupervised-learning>. Accessed on the 22th of August 2025.
- [52] C. Jacobs and S. Poedts. A polytropic model for the solar wind. *Advances in Space Research*, 48(12):1958–1966, 2011.
- [53] J. A. Jacobs et al. Classification of geomagnetic micropulsations. *Journal of Geophysical Research*, 69:180–181, 1964.
- [54] G. James et al. *An Introduction to Statistical Learning with Python*. Springer, Gewerbestrasse 11, Cham, Ch 6330, Switzerland, 2023.
- [55] M.-B. Kallenrode. *Space Physics*. Springer, Gewerbestrasse 11, Cham, Ch 6330, Switzerland, 2003.
- [56] M. Kearns. Thoughts on hypothesis boosting. *Machine Learning Class Project, University of Pennsylvania*, 1988.

- [57] M. Kelley. *The Earth's Ionosphere: Plasma Physics & Electrodynamics*, 2nd edition. Academic Press, 30 Corporate Drive, Suite 400, Burlington, MA 01803, USA, 2009.
- [58] O. Khabarova and C. Price. Importance and challenges of geomagnetic storm forecasting. *Frontiers in Astronomy and Space Science*, 11:85–117, 2024.
- [59] E. Kilpua et al. Geoeffective properties of solar transients and stream interaction regions. *Space Science Reviews*, 212:1271–1314, 2017.
- [60] J. Klockars. <https://github.com/jvkloc/XGBoost-nowcasting-for-SMR-and-Pc5-ULF>. Accessed on the 18th of October 2025.
- [61] D. Knuth. *The Art of Computer Programming Volume 1*, 3rd edition. Addison-Wesley, 221 River Street, Hoboken, New Jersey 07030, USA, 1997.
- [62] S. Krieger and N. Freij. PyCWT: wavelet spectral analysis in Python. V. 0.4.0-beta. Python. 2023. <https://github.com/regeirk/pycwt>. <https://pycwt.readthedocs.io/en/latest/>. Accessed on the 15th of July 2025.
- [63] M. Kuhn and K. Johnson. *Applied Predictive Modeling*. Springer, Gewerbestrasse 11, Cham, Ch 6330, Switzerland, 2013.
- [64] J.-L. Lagrange. Essai sur le problème des trois corps. *Prix de l'Académie Royale des Sciences de Paris, tome IX, in volume 6 of œuvres*, page 292, 1772.
- [65] K. Lau and H. Weng. Climate signal detection using wavelet transform: How to make a time series sing. *Bulletin of the American Meteorological Society*, 76(12).
- [66] Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521:436–444, 2015.
- [67] C. Lee et al. Complex-valued neural networks: A comprehensive survey. *IEEE/CAA Journal of Automatica Sinica*, 9(8):1406 – 1426, 2022.
- [68] V. Lipsanen. Correlation of solar wind parameters and MLT dependent ground-based ULF wave index during large solar wind structures, 2024. Master's thesis, University of Helsinki.
- [69] S. Lundberg. <https://shap.readthedocs.io/en/latest/index.html>. Accessed on the 19th of October 2025.
- [70] S. Lundberg and S.-I. Lee. *Proceedings of the 31st Conference on Neural Information Processing Systems*. Curran Associates Inc., 57 Morehouse Lane, Red Hook, New York, USA, 2017.

- [71] B. Mailyan et al. What is the best method to calculate the solar wind propagation delay? *Annales Geophysicae*, 26:2383–2394, 2013.
- [72] I. Mann et al. Explaining the dynamics of the ultra-relativistic third Van Allen radiation belt. *Nature Physics*, 12:978–983, 2016.
- [73] J. Morlet. Sampling theory and wave propagation. *Issue in Acoustics Signals—Image Processing and Recognition, NATO ASI Series*, 1:233–261, 1983.
- [74] K. Murphy et al. Inner magnetospheric ULF waves: The occurrence and distribution of broadband and discrete wave activity. *Journal of Geophysical Research: Space Physics*, 125, 2020.
- [75] NASA. <https://science.nasa.gov/earth/10-things-to-know-about-the-ionosphere/>. Accessed on the 29th of July 2025.
- [76] NASA. <https://science.nasa.gov/mission/ace/>. Accessed on the 16th of July 2025.
- [77] NASA. <https://science.nasa.gov/mission/wind/>. Accessed on the 16th of July 2025.
- [78] NASA. <https://science.nasa.gov/solar-system/resources/faq/what-are-lagrange-points/>. Accessed on the 1st of August 2025.
- [79] NASA. <https://science.nasa.gov/mission/soho/>. Accessed on the 16th of July 2025.
- [80] P. Newell and J. Gjerloev. Supermag-based partial ring current indices. *Journal of Geophysical Research*, 117(A05215), 2012.
- [81] P. T. Newell et al. A nearly universal solar wind-magnetosphere coupling function inferred from 10 magnetospheric state variables. *Journal of Geophysical Research: Space Physics*, 112, 2007.
- [82] J. Nocedal and S. Wright. *Numerical Optimization, Second Edition*. Springer, Gewerbestrasse 11, Cham, Ch 6330, Switzerland, 2006.
- [83] A. Oppenheim and R. Schaffer. *Discrete-Time Signal Processing*. Pearson Higher Education, One Lake Street, Upper Saddle River, New Jersey 07458, USA, 2009.
- [84] J. Ou, Y. Jiang, and Q. Yan. Study on the local time asymmetry of the earth’s ring currents based on multiple low-earth-orbit satellite magnetic observations. *Science China Technological Sciences*, 67:1–11, 2024.

- [85] E. Parker. Dynamics of the interplanetary gas and magnetic fields. *Astrophysical Journal*, 128:664, 1958.
- [86] F. Pedregosa et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [87] S. Prince. *Understanding Deep Learning*. The MIT Press, 255 Main Street, 9th Floor, Cambridge, Massachusetts 02142, USA, 2024.
- [88] L. Prokhorenkova et al. CatBoost: unbiased boosting with categorical features. *Advances in Neural Information Processing Systems*, 31(4):6638–6648, 2018.
- [89] H. Raju and S. Das. Comparative analysis of various machine-learning models for solar-wind propagation-delay estimation. *Solar Physics*, 299, 2024.
- [90] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65:386–408, 1958.
- [91] D. Rumelhart, G. Hinton, and R. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.
- [92] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach, Global Edition, 4th ed*. Pearson Education Limited, Hockham Way, Harlow, CM17 9SR, United Kingdom, 2022.
- [93] R. Schapire. The strength of weak learnability. *Machine Learning*, 5:197–227, 1990.
- [94] J. Schmidhuber. Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117, 2015.
- [95] V. Serov. *Fourier Series, Fourier Transform and their Applications to Mathematical Physics*. Springer, Gewerbestrasse 11, Cham, Ch 6330, Switzerland, 2017.
- [96] S. Smith. The Scientist and Engineer’s Guide to Digital Signal Processing. <https://www.dspguide.com/ch12/4.htm>. Accessed on the 20th of July 2025.
- [97] Sujatha R. Effects of radiations on satellites exposed to extreme space weather events. *International Journal Of Novel Research And Development*, 9:b156–b173, 2024.
- [98] SuperMAG. <https://supermag.jhuapl.edu/>. Accessed on the 1st of July 2025.
- [99] SuperMAG. <https://supermag.jhuapl.edu/indices/?fidelity=low&layers=SME.UL&start=2001-01-29T22%3A00%3A00.000Z&step=14400&tab=download>. Accessed on the 18th of July 2025.

- [100] SuperMAG. <https://supermag.jhuapl.edu/mag/?fidelity=low&start=2001-01-01T00%3A00%3A00.000Z&interval=23%3A59&tab=customdownload>. Accessed on the 18th of July 2025.
- [101] SuperMAG web API. <https://supermag.jhuapl.edu/mag/?fidelity=low&start=2001-01-01T00%3A00%3A00.000Z&interval=23%3A59&tab=api>. Accessed on the 18th of July 2025.
- [102] N. Tatti. Approximating splits for decision trees quickly in sparse data streams. *2025 SIAM International Conference on Data Mining, SDM 2025. Society for Industrial and Applied Mathematics Publications*, pages 647–655, 2025.
- [103] The Arctic University of Norway. https://en.uit.no/education/program/270600/space_physics. Accessed on the 15th of August 2025.
- [104] R. Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996.
- [105] C. Torrence and G. Compo. A practical guide to wavelet analysis. *Bulletin of the American Meteorological Society*, 79(1):61–78, 1998.
- [106] M. Trunk. A problem of dimensionality: A simple example. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1(3):306–307, 1979.
- [107] University of Oulu. <https://www.oulu.fi/en/university/faculties-and-units/faculty-science/space-physics-and-astronomy>. Accessed on the 15th of August 2025.
- [108] L. Valiant. A theory of the learnable. *Communications of the ACM*, 27(11):1134–1142, 1984.
- [109] G. van Rossum. Python tutorial. *Technical Report CS-R9526, Centrum voor Wiskunde en Informatica, Amsterdam*, 1995. <https://www.python.org/>. Accessed on the 9th of August 2025.
- [110] J. Vitter. *Algorithms and Data Structures for External Memory*. Now Publishers Inc., PO Box 1024, Hanover, Massachusetts 02339, USA, 2008.
- [111] Y. Wang et al. *Guidelines for Nowcasting Techniques*. World Meteorological Organization, 7bis, avenue de la Paix, CH-1211 Geneva 2, Switzerland, 2017.

Appendix A. Mathematics

Definitions of the Taylor polynomial, complex conjugate, and convolution and the convolution theorem are presented here. Taylor polynomial is used in XGBoost's objective function. Complex conjugate and the convolution theorem are used in the wavelet computation.

A.1 Taylor polynomial

The n -th order Taylor polynomial approximating function f around point a is defined as

$$f(x) \approx \sum_{n=0}^n \frac{f^{(n)}(a)}{n!} (x-a)^n,$$

so the second order Taylor polynomial is

$$f(x) \approx f(a) + f'(a)(x-a) + \frac{1}{2}f''(a)(x-a)^2. \quad [6]$$

In the context of XGBoost, the loss function $l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i))$ is approximated around the previous prediction $\hat{y}_i^{(t-1)}$ and l is a function of the prediction $\hat{y}_i$ so $x-a$ is $\hat{y}_i^{(t-1)} + f_t(x_i) - \hat{y}_i^{(t-1)} = f_t(x_i)$. Thus the second order Taylor polynomial of l is

$$l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) \approx l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i (f_t(x_i))^2,$$

where $g_i = \partial_{\hat{y}_i^{(t-1)}} l(y_i, \hat{y}_i^{(t-1)})$ is the gradient and $h_i = \partial_{\hat{y}_i^{(t-1)}}^2 l(y_i, \hat{y}_i^{(t-1)})$ is the Hessian.

A.2 Complex conjugate

Complex numbers are defined as $x + iy$, where $x, y \in \mathbb{R}$ and i is the imaginary number defined as $i = \sqrt{-1}$. The set of all complex numbers is denoted by $\mathbb{C}$. The complex conjugate of a complex number $z = x + iy$ is $\bar{z} = x - iy$ and it has the following properties

$$\overline{z + w} = \bar{z} + \bar{w} \text{ and } \overline{zw} = \bar{z}\bar{w}$$

for $z, w \in \mathbb{C}$. [4].

A.3 Convolution theorem

The convolution $f * g$ of two functions $f(x), g(x)$, $x \in \mathbb{R}$, is given by

$$(f * g)(x) = \int_{-\infty}^{\infty} f(x-y)g(y) dy$$

and convolution for discrete and finite sequences $\{x_n\}$, $\{y_n\}$, $n \in \{0, 1, \dots, N-1\}$ is

$$(x * h)_n = \sum_{m=0}^{N-1} x_{n-m} y_m. \quad [28]$$

The convolution theorem states that the Fourier transform $\mathcal{F}$ of the convolution of $\{x_n\}$, $\{y_n\}$, $n \in \{0, 1, \dots, N-1\}$ is the product of their Fourier transforms:

$$\mathcal{F}(x * y) = \mathcal{F}\{x\} \mathcal{F}\{y\}. \quad [83]$$

In the context of the wavelet transform on discrete sequences, the convolution theorem states that the Fourier transform of Equation 3.10 is the product of Fourier transforms of the sequence x_n and the function ψ^* .

Appendix B. Performance metrics

Root mean squared error (RMSE), mean absolute error (MAE), and median absolute error (MdAE) are presented in this appendix, in addition to scaled versions of them from [50]. These metrics were used in XGBoost model performance evaluation. Root mean squared error is defined as

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2},$$

where m is the validation or test set size, y is the true target value of the i -th sample and $\hat{y}_i$ is the predicted target value of the i -th sample. In addition to the RMSE, a scaled version of it, RMSSE, was used. It was obtained by scaling each of the prediction errors by the RMSE of a naive forecaster predicting the previous value:

$$\text{RMSSE} = \sqrt{\frac{1}{m} \sum_{t=1}^m e_t^2},$$

where

$$e_t = \frac{y_t - \hat{y}_t}{\sqrt{\frac{1}{n-1} \sum_{i=2}^n (y_i - y_{i-1})^2}},$$

in which n is the training set size. An RMSSE value less than 1 indicates that the model's total error is less than the trivial last value forecaster's total error. Smaller RMSSE values indicate better performance. The RMSSE is equal to 1 if and only if the total error of the model is equal to the total error of the naive forecaster. A value above 1 indicates that the naive forecaster outperforms the model considering the total error, which may suggest overfitting or that the naive model is already very strong for the data.

Mean absolute error is defined as

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i|$$

and median absolute error is

$$\text{MdAE} = \text{median}(\{|y_i - \hat{y}_i| : i \in \{1, 2, \dots, m\}\}).$$

MASE and MdASE are defined and interpreted similarly to the RMSSE:

$$\text{MASE} = \frac{1}{m} \sum_{t=1}^m |e_t|$$

where

$$e_t = \frac{y_t - \hat{y}_t}{\frac{1}{n-1} \sum_{i=2}^n |y_i - y_{i-1}|},$$

and

$$\text{MdASE} = \text{median}(\{|e_t| : t \in \{1, 2, \dots, m\}\})$$

where e_t is the same as in MASE.

RMSE increases with large errors more than MAE because of squaring, making it sensitive to outliers. The models were trained with an RMSE objective function, emphasizing performance on extreme events. MAE measures the average absolute deviation between predictions and the ground truth. It is easier to interpret directly as it tells the actual average error. It is less sensitive to outliers than RMSE. MdAE is the median absolute deviation from true values representing the typical nowcast error in normal conditions. It was included as a baseline to compare RMSE and MAE to.

Appendix C. Neural Networks and Deep Learning

Neural networks refer to machine learning models inspired by the structure and function of the human brain. They consist of interconnected processing units, often called neurons, that are arranged in layers. Each neuron computes a weighted sum of its inputs, applies a nonlinear activation function, and passes the result forward. By adjusting the connection weights during training, neural networks can learn to approximate complex functions from data. [35]

Deep learning refers to the use of neural networks with many hidden layers. This depth enables the network to learn hierarchical feature representations: lower layers typically capture simple patterns, while deeper layers capture increasingly abstract structures. This ability to learn directly from raw data without hand-crafted features distinguishes deep learning from more traditional machine learning approaches. [66]

Early models such as the perceptron [90] introduced the idea of a trainable neural unit, but were limited in their expressiveness. The breakthrough came in the 1980s with the rediscovery and popularization of the backpropagation algorithm [91] which enabled multilayer network training in practice. In the 2000s and 2010s, with increased computational power and larger datasets, these techniques evolved into what is now called deep learning [94].

Appendix D. Feature importance and SHAP summary

XGBoost’s gain feature importance scores (Section 5.1) and a short description of Shapley Additive Explanations are given in this appendix.

D.1 XGBoost feature importance

Tables in this section list the feature importance of the models predicting SMR indices in column-major order.

Table D.1: SMR-00 feature importance.

Feature	Score	Feature	Score
SMR00_lag1	0.4292	BGSM_z_lag2	0.0067
SMR00_lag2	0.2183	SC_pos_GSM_Re_y_lag1	0.0062
SMR00_lag4	0.0612	BGSM_z	0.0057
SMR00_lag3	0.0297	SMR18_lag4	0.0056
Magnitude	0.0182	SMR06_lag1	0.0056
SC_pos_GSM_Re_x_lag2	0.0159	SMR12_lag2	0.0055
Magnitude_lag1	0.0151	BGSM_z_lag1	0.0053
Magnitude_lag4	0.0149	V_GSM_y	0.0047
Magnitude_lag3	0.0131	BGSM_z_lag4	0.0041
SMR12_lag1	0.0112	SMR12_lag3	0.0041
SC_pos_GSM_Re_x_lag3	0.0110	unix_time	0.0038
Magnitude_lag2	0.0101	BGSM_z_lag3	0.0034
SC_pos_GSM_Re_x	0.0095	SMR06_lag2	0.0033
SMR_lag1	0.0083	SMR12_lag4	0.0033
SC_pos_GSM_Re_x_lag1	0.0075	SMR18_lag2	0.0032
SMR18_lag1	0.0073	SMR06_lag4	0.0021
SC_pos_GSM_Re_y_lag2	0.0073	SMR18_lag3	0.0013
SC_pos_GSM_Re_x_lag4	0.0072	SMR06_lag3	0.0012
Tpr	0.0072	SMR_lag2	0.0005
SC_pos_GSM_Re_y_lag3	0.0072	SMR_lag3	0.0005
SC_pos_GSM_Re_y	0.0070	SMR_lag4	0.0004
SC_pos_GSM_Re_y_lag4	0.0069		

Table D.2: SMR-06 feature importance.

Feature	Score	Feature	Score
SMR06_lag1	0.3139	Magnitude_lag3	0.0055
SMR06_lag2	0.2144	SC_pos_GSM_Re_y_lag3	0.0054
SMR06_lag3	0.0849	Magnitude_lag4	0.0050
SMR_lag4	0.0813	BGSM_z_lag4	0.0048
SMR06_lag4	0.0682	V_GSM_y	0.0045
SMR18_lag1	0.0207	SC_pos_GSM_Re_y_lag1	0.0044
SMR_lag1	0.0205	SMR00_lag4	0.0043
Magnitude	0.0107	Magnitude_lag2	0.0040
SMR18_lag4	0.0105	SC_pos_GSM_Re_x	0.0040
SC_pos_GSM_Re_x_lag3	0.0098	BGSM_z_lag3	0.0039
unix_time	0.0097	BGSM_z_lag1	0.0037
SMR18_lag2	0.0088	SC_pos_GSM_Re_y_lag2	0.0037
SMR18_lag3	0.0087	SMR12_lag3	0.0036
SC_pos_GSM_Re_x_lag2	0.0085	BGSM_z_lag2	0.0036
SMR_lag3	0.0083	BGSM_z	0.0032
SC_pos_GSM_Re_x_lag4	0.0080	SMR12_lag4	0.0028
SMR12_lag1	0.0077	SMR00_lag1	0.0028
Magnitude_lag1	0.0075	SMR00_lag3	0.0011
SC_pos_GSM_Re_x_lag1	0.0066	SMR00_lag2	0.0008
SC_pos_GSM_Re_y	0.0065	SMR12_lag2	0.0006
SC_pos_GSM_Re_y_lag4	0.0064	SMR_lag2	0.0006
Tpr	0.0060		

Table D.3: SMR-12 feature importance.

Feature	Score	Feature	Score
SMR12_lag1	0.4834	SC_pos_GSM_Re_x_lag1	0.0038
SMR12_lag2	0.2232	SC_pos_GSM_Re_y	0.0037
SMR12_lag4	0.0683	SMR18_lag2	0.0032
SMR12_lag3	0.0598	SC_pos_GSM_Re_x_lag3	0.0032
Magnitude	0.0129	SC_pos_GSM_Re_y_lag1	0.0031
Magnitude_lag1	0.0102	SMR00_lag1	0.0029
Magnitude_lag2	0.0102	Tpr	0.0028
SMR18_lag1	0.0085	unix_time	0.0026
Magnitude_lag3	0.0084	SMR18_lag3	0.0025
Magnitude_lag4	0.0084	BGSM_z_lag4	0.0025
SC_pos_GSM_Re_y_lag3	0.0079	SMR06_lag4	0.0023
SC_pos_GSM_Re_y_lag4	0.0068	SC_pos_GSM_Re_x_lag4	0.0022
BGSM_z	0.0060	V_GSM_y	0.0021
SC_pos_GSM_Re_x_lag2	0.0053	BGSM_z_lag3	0.0020
SMR18_lag4	0.0048	SMR00_lag4	0.0017
BGSM_z_lag2	0.0048	SMR06_lag2	0.0016
SMR_lag1	0.0046	SMR00_lag3	0.0013
BGSM_z_lag1	0.0045	SMR_lag4	0.0011
SC_pos_GSM_Re_x	0.0042	SMR06_lag3	0.0006
SC_pos_GSM_Re_y_lag2	0.0041	SMR00_lag2	0.0005
SMR_lag2	0.0040	SMR_lag3	0.0002
SMR06_lag1	0.0039		

Table D.4: SMR-18 feature importance.

Feature	Score	Feature	Score
SMR18_lag2	0.3451	SC_pos_GSM_Re_y_lag2	0.0034
SMR18_lag1	0.3120	SC_pos_GSM_Re_x_lag3	0.0032
SMR18_lag3	0.1036	SMR06_lag4	0.0030
SMR18_lag4	0.0680	SC_pos_GSM_Re_y_lag1	0.0027
Magnitude	0.0130	SC_pos_GSM_Re_x_lag1	0.0027
SMR00_lag1	0.0123	SC_pos_GSM_Re_x	0.0026
Magnitude_lag1	0.0106	BGSM_z_lag4	0.0025
Magnitude_lag3	0.0103	SMR06_lag1	0.0024
Magnitude_lag2	0.0099	BGSM_z_lag2	0.0023
Magnitude_lag4	0.0098	SC_pos_GSM_Re_y	0.0023
SMR12_lag1	0.0081	SMR12_lag3	0.0022
SMR00_lag3	0.0064	SC_pos_GSM_Re_x_lag4	0.0020
SMR12_lag4	0.0062	SMR00_lag4	0.0018
SMR12_lag2	0.0060	Tpr	0.0017
SMR_lag1	0.0059	unix_time	0.0016
SMR_lag2	0.0058	V_GSM_y	0.0014
BGSM_z	0.0054	SMR06_lag2	0.0008
BGSM_z_lag3	0.0046	SMR00_lag2	0.0007
SC_pos_GSM_Re_y_lag3	0.0045	SMR_lag3	0.0006
SC_pos_GSM_Re_y_lag4	0.0043	SMR06_lag3	0.0006
BGSM_z_lag1	0.0039	SMR_lag4	0.0004
SC_pos_GSM_Re_x_lag2	0.0038		

Table D.5: SMR global feature importance.

Feature	Score	Feature	Score
SMR_lag1	0.2557	SC_pos_GSM_Re_y_lag3	0.0055
SMR_lag2	0.2503	SC_pos_GSM_Re_x_lag2	0.0050
SMR_lag4	0.0984	SMR12_lag3	0.0046
SMR00_lag4	0.0550	SC_pos_GSM_Re_x	0.0044
SMR_lag3	0.0435	SC_pos_GSM_Re_x_lag1	0.0041
SMR18_lag4	0.0268	SMR00_lag3	0.0041
SMR00_lag1	0.0266	SC_pos_GSM_Re_x_lag3	0.0040
SMR00_lag2	0.0251	SC_pos_GSM_Re_x_lag4	0.0039
SMR18_lag2	0.0214	SC_pos_GSM_Re_y_lag1	0.0036
Magnitude	0.0169	SC_pos_GSM_Re_y_lag2	0.0035
Magnitude_lag3	0.0138	BGSM_z	0.0035
Magnitude_lag1	0.0134	Tpr	0.0034
Magnitude_lag4	0.0117	SC_pos_GSM_Re_y_lag4	0.0032
SMR12_lag4	0.0116	BGSM_z_lag4	0.0029
SMR18_lag1	0.0110	SMR18_lag3	0.0025
Magnitude_lag2	0.0101	unix_time	0.0024
SMR06_lag2	0.0079	SMR12_lag2	0.0019
SMR12_lag1	0.0070	SC_pos_GSM_Re_y	0.0018
SMR06_lag1	0.0068	BGSM_z_lag3	0.0017
BGSM_z_lag2	0.0068	V_GSM_y	0.0017
SMR06_lag4	0.0061	SMR06_lag3	0.0006
BGSM_z_lag1	0.0057		

D.2 Shapley Additive Explanations

Shapley Additive Explanations (SHAP) is a game-theoretic approach for explaining the output of any machine learning model. While XGBoost’s feature importance uses one of the three metrics (Section 5.1) based on the overall model performance for scoring features, SHAP analysis reveals the contribution of every feature for each individual prediction without considering the model performance. In addition to magnitude, SHAP analysis includes the direction of influence of each feature value for every prediction.

A feature with high XGBoost gain feature importance score is used in splits that reduce error the most. It could be driving one prediction higher and another lower, but XGBoost’s feature importance does not say anything about that. A feature used in very many splits has a high weight feature importance score, but XGBoost does not report

anything about the actual influence of the feature. The high score simply means that the feature is present in many splits. XGBoost feature importances may give different scores to a single feature. SHAP analysis provides consistency as well as accuracy compared to XGBoost's inbuilt feature importance.

Figure D.1 on the next page shows SMR-18 model feature summary as an example. Red dots on the positive side indicate that high values of the feature increase the predicted value and red dots on the left indicate that high values of the feature decrease it. Similarly for low values of features marked with blue dots: blue on the right indicates low values of the feature increasing the prediction and blue values on the left indicate that low values of the feature decrease the prediction. In this case there are hundreds of thousands of values so the complete summary plot is not suitable for detailed analysis.

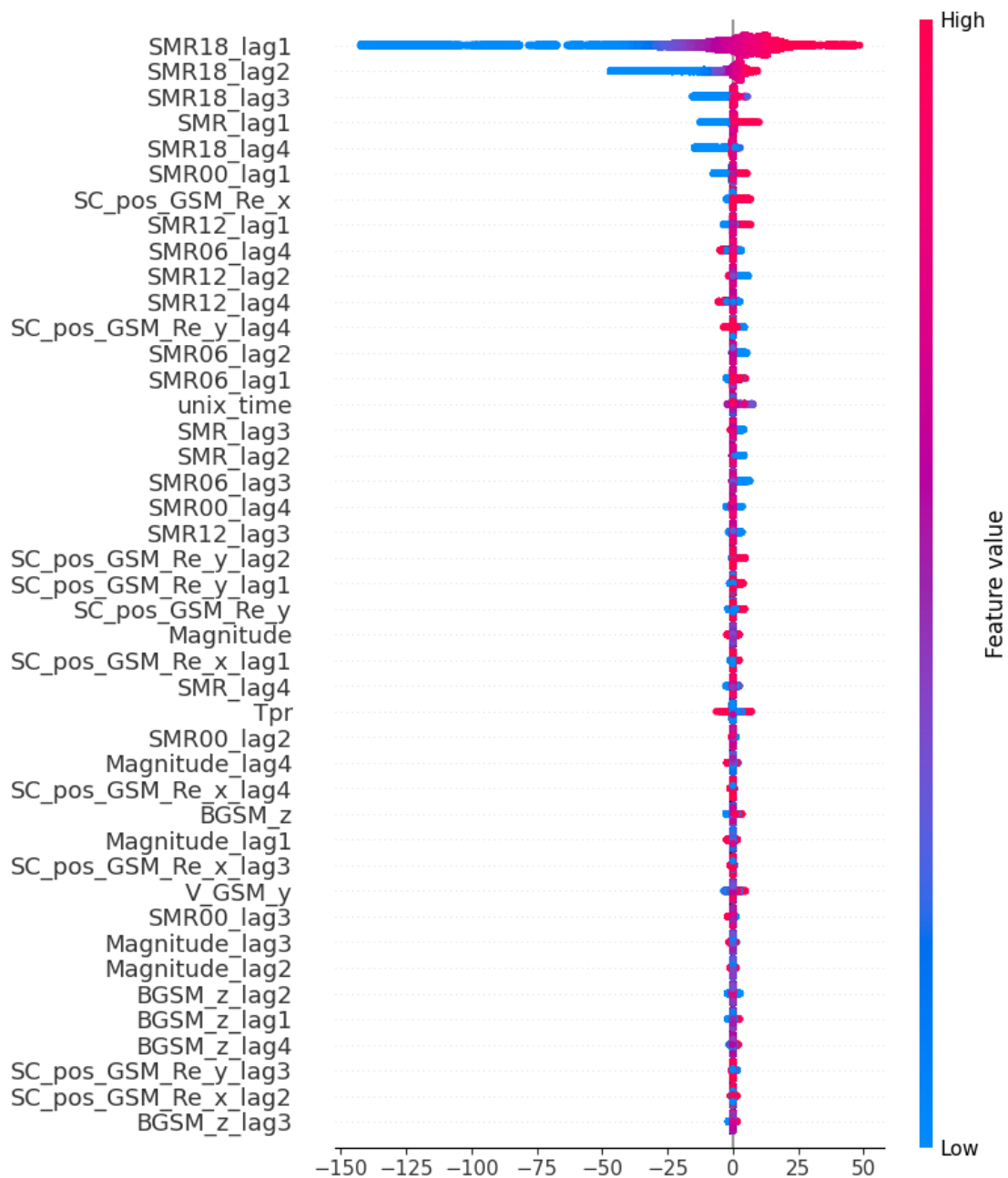


Figure D.1: SHAP summary for SMR-18 model.